

Control de referencia — clases 1 a 10

Introducción a la Ciencia de Datos · Universidad de Montevideo — FCEE

Índice

Clase 1 — Git y trabajo reproducible	1
Clase 2 — El proceso generador de datos	2
Clase 3 — Objetos, tipos y errores	3
Paquetes	4
Clase 4 — Rutas, carga y perfilado	6
Clase 5 — Flujo de análisis y formatos	7
Clase 6 — Medición y diseño de encuestas	7
Clase 7 — Carga, limpieza, fechas y agregación	8
Clase 8 — Visualización	9
Clase 9 — Unión de fuentes y gráficos por grupo	11
Clase 10 — Series de tiempo: una primera mirada	12
Lecturas obligatorias	12
Clave de corrección	14

Cada pregunta tiene cuatro opciones. Marcar **cada opción** como verdadera (V) o falsa (F): pueden ser verdaderas una, varias, todas o ninguna. La clave de corrección está al final.

Clase 1 — Git y trabajo reproducible

1. Una carpeta con TrabajoFinal_v2_FINAL_corregido_AHORA_SI.xlsx. Versionar resuelve:

- a) Saber cuál es la versión buena
- b) Saber qué cambió entre una versión y otra
- c) Saber quién lo cambió y por qué
- d) Impedir que dos personas trabajen a la vez

2. Una confirmación (*commit*):

- a) Registra quién hizo el cambio, cuándo y por qué
- b) Es una “foto” del estado de los archivos en ese momento
- c) Se pierde al apagar la computadora
- d) Permite volver a esa versión más adelante

3. Ana editó `bitacora.md` en VS Code, guardó con `Ctrl+S` y cerró la computadora sin hacer nada más:

- a) El cambio está guardado en su máquina
- b) El cambio ya quedó en el historial
- c) El equipo lo puede ver en GitHub
- d) Para que quede registrado falta confirmar y sincronizar

4. El mensaje de confirmación:

- a) “cambios” es un mensaje adecuado
- b) “limpieza: elimina duplicados de negocio en ventas” es un mensaje adecuado
- c) Debe decir por qué se hizo el cambio
- d) Puede quedar vacío

5. ¿Qué se versiona?

- a) El código (`.py`, `.R`, `.qmd`)
- b) Los datos crudos
- c) El script que obtiene los datos
- d) Las contraseñas, en un archivo de configuración

6. Al sincronizar, la interfaz avisa que hay un conflicto en `decisiones.md`:

- a) El repositorio se rompió y hay que clonarlo de nuevo
- b) Otra persona modificó las mismas líneas del mismo archivo
- c) Hay que elegir qué versión queda y volver a confirmar
- d) Conviene conversarlo en el equipo y anotar la decisión

7. Repositorios y autoría en el curso:

- a) El equipo tiene un repositorio compartido y cada estudiante una bitácora propia
- b) Cada estudiante confirma desde su propia cuenta
- c) Alcanza con que una persona suba el trabajo de todos
- d) El docente ve el trabajo cuando ocurre, no solo en la entrega

8. En el repositorio del equipo:

- a) `bitacora.md` lleva una entrada por semana: qué intentamos, qué falló, qué decidimos
- b) Una semana sin nada que haya fallado es una buena semana
- c) `decisiones.md` registra cada decisión con la alternativa que se descartó
- d) Los datos crudos se suben a la carpeta `datos/`

Clase 2 — El proceso generador de datos

9. El Proceso Generador de Datos (DGP):

- a) Es el archivo `.csv` que recibimos
- b) Es el mecanismo que produce los eventos que intentamos registrar
- c) Es la “caja negra” de Breiman
- d) El modelo lo conoce automáticamente al ajustarse

10. Las variables de la dinámica de la caja negra:

- a) Textura (suave / rugoso / poroso) es nominal
- b) Rigidez (blando / medio / rígido) es ordinal
- c) Cantidad de aristas es numérica continua
- d) Peso estimado en gramos es numérica continua

11. Datos sintéticos y *bootstrap*:

- a) El *bootstrap* es muestreo con reemplazo de la muestra observada
- b) `np.random.seed(42)` hace reproducible la generación aleatoria
- c) Generar con una normal siempre representa bien lo que hay en la caja
- d) Los datos sintéticos permiten compartir datos sin exponer el original

12. “*Garbage in, garbage out*”:

- a) El modelo corrige los errores de medición
 - b) El modelo aprende y magnifica los sesgos de los datos
 - c) Los datos sintéticos compensan un dataset base pobre
 - d) La riqueza del DGP limita lo que un modelo puede aprender
-

Clase 3 — Objetos, tipos y errores

13. ¿Qué es la vectorización?

- a) Que una operación se aplica a todos los elementos a la vez, sin escribir un bucle
- b) En R es el comportamiento por defecto: `c(1, 2, 3) * 10` da 10 20 30
- c) En Python, `[1, 2, 3] * 10` vectoriza igual que en R
- d) En Python se obtiene con `np.array([1, 2, 3]) * 10`

14. ¿Qué es un vector?

- a) En R, lo que devuelve `c()`
- b) En Python, `[]` es un vector con el mismo comportamiento que en R
- c) Una secuencia ordenada de elementos del mismo tipo
- d) En R, el número 1 escrito solo

15. La asignación:

- a) `edad = 35` es una ecuación
- b) `edad = edad + 1` es válido
- c) `<-` en R cumple el rol de `=` en Python
- d) Al reasignar un nombre, el valor anterior se pierde

16. Tipos elementales:

- a) `"45.0"` es un float
- b) `"45.0" + "1"` devuelve `'45.01'`
- c) `True` / `TRUE` es el tipo lógico
- d) `NA` en R y `NaN` / `None` en Python representan ausencia

17. Los errores del primer día:

- a) `NameError` significa que el archivo no existe
- b) `TypeError` significa que la operación no corresponde a ese tipo
- c) `FileNotFoundError` es un error de ubicación
- d) Un mensaje de error se lee de abajo hacia arriba

18. `c(1, "a", TRUE)` en R:

- a) Da error por mezclar tipos
- b) Devuelve un vector de texto: "1" "a" "TRUE"
- c) La conversión es silenciosa
- d) Tiene longitud 3

19. Un data frame:

- a) Es una colección de columnas
- b) Cada columna tiene su propio tipo
- c) Cada fila tiene un único tipo
- d) Admite columnas de distinto largo

20. `df["peso"] > 50`:

- a) Devuelve un único `True`
- b) Devuelve un `True` o `False` por cada fila
- c) `df[df["peso"] > 50]` filtra las filas `True`
- d) `(df["peso"] > 50).sum()` cuenta cuántas son `True`

21. Función y método:

- a) `len(x)` es una función aplicada al objeto
- b) `df.head()` es un método del objeto
- c) R usa métodos con punto todo el tiempo
- d) `length(x)` es un método

22. En `pd.read_csv("datos.csv", sep=";")`:

- a) `sep=";"` es un dato más que se lee
- b) Cambiar `sep` cambia el archivo
- c) El mismo archivo leído con dos separadores produce dos tablas distintas
- d) El primer argumento es el dato; los demás son instrucciones

23. Índices:

- a) En R, el primer elemento es `x[1]`
- b) En Python, el primer elemento es `x[0]`
- c) En Python, `x[1]` es el primer elemento
- d) En R, `x[0]` es el primer elemento

Paquetes

24. ¿Qué es un paquete?

- a) Un conjunto de funciones que otros ya escribieron y que se agrega al lenguaje
- b) Viene siempre incluido al instalar Python o R

- c) Se instala una sola vez y se carga en cada script
- d) requirements.txt lista los paquetes que necesita el proyecto

25. Instalar y cargar:

- a) pip install pandas se ejecuta en la terminal, una sola vez
- b) import pandas as pd se escribe al principio de cada script
- c) install.packages("ggplot2") se corre cada vez que se abre RStudio
- d) library(ggplot2) carga el paquete en la sesión

26. ¿Cuáles son nombres correctos de paquetes de Python?

- a) numpy
- b) pynump
- c) pandas
- d) matplotlib

27. ¿Cuáles son nombres correctos de paquetes de R?

- a) ggplot2
- b) tidyverse
- c) pandas
- d) lubridate

28. En Python:

- a) pandas maneja tablas (DataFrame): leer CSV, filtrar, agrupar
- b) numpy maneja arrays numéricos y vectoriza
- c) matplotlib hace gráficos
- d) pathlib hace gráficos

29. En R:

- a) readr lee archivos CSV
- b) dplyr filtra, agrupa y resume tablas
- c) ggplot2 lee archivos Excel
- d) lubridate maneja fechas y here construye rutas

30. Aparece No module named 'pandas' (Python) o there is no package called 'ggplot2' (R):

- a) Hay un error de sintaxis en el script
- b) El paquete no está instalado en ese entorno
- c) Se resuelve con pip install / install.packages()
- d) Se resuelve reescribiendo la línea import / library()

31. import pandas as pd:

- a) pd es un alias para escribir menos
- b) Después se usa pd.read_csv(...)
- c) import pandas sin alias también funciona; después se escribe pandas.read_csv(...)
- d) El alias pd es obligatorio

Clase 4 — Rutas, carga y perfilado

32. Rutas:

- a) "data/raw/datos.csv" es una ruta relativa
- b) "C:/Users/jodavyt/..." es absoluta y falla en otra máquina
- c) La ruta absoluta es más reproducible
- d) Para el lenguaje, una ruta es texto

33. Directorio de trabajo:

- a) `getwd()` dice dónde está parado el entorno
- b) `os.getcwd()` es el equivalente en Python
- c) `setwd()` genera rutas relativas
- d) `here()` construye rutas desde la raíz del proyecto

34. `pathlib`:

- a) `Path.cwd()` da el directorio actual
- b) El operador `/` une partes de una ruta
- c) `ruta.exists()` verifica que el archivo esté
- d) Solo funciona en Windows

35. Perfilado:

- a) `df.info()` equivale a `str(df)`
- b) `df["col"].nunique()` equivale a `length(unique(df$col))`
- c) `df.shape` devuelve (filas, columnas)
- d) `df.describe()` cuenta duplicados

36. El identificador:

- a) Es una columna con tantos valores únicos como filas
- b) Siempre es la primera columna
- c) Sirve para unir tablas
- d) Puede ser compuesto, por ejemplo fecha + estación

37. Faltantes y duplicados:

- a) `df.isnull().sum()` cuenta faltantes por columna
- b) "" y "N/A" se cuentan como faltantes automáticamente
- c) `df.duplicated().sum()` cuenta filas exactamente iguales
- d) `df.drop_duplicates()` las elimina

38. La unidad de observación:

- a) Es lo que representa cada fila
 - b) Partida, jugador y título son unidades distintas de un mismo dominio
 - c) No importa para unir tablas
 - d) Un mismo problema puede tener varios niveles de unidad
-

Clase 5 — Flujo de análisis y formatos

39. El flujo de análisis:

- a) Va de ingesta → perfilado → limpieza → exploración → transformación → entrega
- b) El perfilado se hace después de limpiar
- c) El perfilado incluye nulos, cardinalidad y valores fuera de rango
- d) En la ingesta no se modifican los datos originales

40. Formatos:

- a) CSV guarda el tipo de dato de cada columna
- b) Parquet es binario, columnar y comprimido
- c) JSON es clave-valor y es el estándar de las APIs
- d) Excel tiene un máximo de filas por hoja

41. Excel y PostgreSQL:

- a) Excel fuerza integridad referencial entre tablas
- b) PostgreSQL exige un tipo por columna y admite claves primarias y foráneas
- c) Excel es una base de datos relacional
- d) PostgreSQL maneja muchas conexiones simultáneas

42. El bloque try / except UnicodeDecodeError:

- a) Intenta leer en UTF-8 y, si falla, reintenta con latin-1
- b) UnicodeDecodeError es un error de tipo
- c) El seudocódigo diseña la lógica antes de escribir la sintaxis
- d) El except borra el archivo

43. Tipos de carga:

- a) La carga inicial (*full load*) lee todo el volumen
- b) La carga incremental lee solo lo nuevo o modificado
- c) La incremental es más simple de implementar
- d) CDC (*change data capture*) es una estrategia incremental

44. Filtrar temperaturas mayores a 20:

- a) En R: `temperaturas[temperaturas > 20]`
 - b) En Python: `[t for t in temperaturas if t > 20]`
 - c) En R, `temperaturas > 20` devuelve los valores mayores a 20
 - d) Con `c(18, 22, 15, 25, 30)` el resultado es `22 25 30`
-

Clase 6 — Medición y diseño de encuestas

45. Niveles de medición:

- a) Los niveles de Stevens son nominal, ordinal, intervalo y razón
- b) La temperatura en °C es de razón
- c) El ingreso en pesos es de razón
- d) Satisfacción de 1 a 5 es ordinal

46. Formas de generar datos:

- a) Una respuesta en una encuesta es preferencia declarada
- b) Una compra registrada es preferencia revelada
- c) La preferencia declarada es siempre más confiable
- d) La huella digital es un registro automático de la conducta

47. Datos sensibles (Ley 18.331):

- a) La salud es dato sensible
- b) La ideología política es dato sensible
- c) El precio de un producto es dato sensible
- d) Hay riesgo de reidentificación aunque se quite el nombre

48. El instrumento:

- a) Validez: mide lo que dice medir
- b) Confiabilidad: da resultados consistentes al repetir
- c) Validez y confiabilidad son sinónimos
- d) Operacionalizar es traducir un concepto en variables medibles

49. Faltantes en una encuesta:

- a) Faltante estructural: la pregunta no aplicaba
 - b) No respuesta: aplicaba y no se contestó
 - c) Los dos se tratan igual
 - d) El libro de códigos documenta cada variable y sus valores
-

Clase 7 — Carga, limpieza, fechas y agregación

50. Codificación:

- a) `encoding="latin1"` es una instrucción de lectura
- b) Ver "Ã³" en lugar de "ó" indica un desajuste de codificación
- c) Cambiar la codificación modifica el archivo
- d) `str.replace / str_replace_all` con un diccionario limpia los nombres de estación

51. Fechas:

- a) `read_csv` de `readr` detecta fechas automáticamente
- b) `pandas` necesita `parse_dates=["fecha"]`
- c) `POSIXct` guarda la fecha como texto
- d) El accesor `.dt` cumple el rol de `lubridate`

52. Faltantes:

- a) `dropna()` quita filas con NaN en cualquier columna
- b) `dropna(subset=["o3"])` mira solo la columna `o3`
- c) `filter(!is.na(o3))` es el equivalente en R
- d) `na.omit()` mira una sola columna

53. Agregación:

- a) `group_by()` + `summarise()` equivale a `groupby()` + `agg()`
- b) `n()` en R y "size" en pandas cuentan filas
- c) `floor_date()` + `group_by()` equivale a `pd.Grouper(freq=...)`
- d) `mean()` en R ignora los NA por defecto

54. Al agregar de minutal a horario:

- a) La media representa el nivel típico
- b) El máximo es sensible a errores del sensor
- c) El percentil 95 es robusto a picos aislados
- d) `n_min` dice cuántos minutos respaldan el valor

55. Faltantes en una serie de sensor:

- a) Explícito: la fila existe y la medición es NA
- b) Implícito: la fila no existe
- c) En rachas: muchos NA seguidos, sensor caído
- d) Eliminar los NA sin mirar no pierde información

56. Histograma y tabla de frecuencias:

- a) La densidad es $h_i = f_i / a_i$
 - b) `hist()` en R usa intervalos (a, b] por defecto
 - c) `right = FALSE` los cambia a [a, b)
 - d) La media calculada desde la tabla es exacta
-

Clase 8 — Visualización

57. Partes de un gráfico:

- a) Dataset, variable x, variable y, tipo de gráfico y títulos
- b) "Valores" es un buen rótulo para el eje y
- c) El rótulo del eje debe incluir la unidad
- d) `ylim(0, 40)` es una decisión de lectura, no de datos

58. ¿Qué tipo de gráfico para qué pregunta?

- a) Una variable a lo largo del tiempo: línea
- b) Comparar un valor entre categorías (estaciones): barras
- c) La relación entre dos variables numéricas: dispersión
- d) La distribución de una variable: línea

59. Histograma y boxplot:

- a) El histograma cuenta cuántas observaciones caen en cada intervalo
- b) El boxplot muestra mediana, cuartiles y valores extremos
- c) Los dos muestran la distribución de una variable, no su evolución en el tiempo
- d) El histograma necesita dos variables

60. En `ggplot2`:

- a) `ggplot(datos, aes(x = mes, y = o3_medio_mes))` define los datos y qué va en cada eje

- b) `+ geom_line()` agrega una capa
- c) `labs(title = ..., x = ..., y = ...)` pone los títulos
- d) `geom_line()` solo, sin `ggplot()` antes, produce el gráfico

61. En matplotlib:

- a) `fig, ax = plt.subplots()` crea la figura y los ejes
- b) `ax.plot(x, y)` dibuja una línea
- c) `ax.scatter(x, y)` dibuja puntos
- d) `ax.set(title = ..., xlabel = ..., ylabel = ...)` pone los títulos

62. Equivalencias entre ggplot2 y matplotlib:

- a) `geom_line()` ↔ `ax.plot()`
- b) `geom_point()` ↔ `ax.scatter()`
- c) `geom_histogram()` ↔ `ax.hist()`
- d) `geom_boxplot()` ↔ `ax.plot()`

63. Capas:

- a) `geom_line()` + `geom_point()` son dos capas sobre los mismos datos
- b) `ax.plot()` + `ax.scatter()` es el equivalente en matplotlib
- c) El orden de las capas no cambia lo que se ve
- d) `theme_minimal()` cambia los datos

64. Ejes y escalas, en el gráfico de ozono mensual:

- a) Un eje x con 1, 2, 3, 4 se lee mejor con Ene, Feb, Mar, Abr
- b) Empezar el eje y en 25 en lugar de 0 exagera las diferencias entre meses
- c) `ax.set_xticks([1, 2, 3, 4], ["Ene", "Feb", "Mar", "Abr"])` cambia las etiquetas del eje
- d) Cambiar los límites del eje cambia los datos

65. Títulos y fuente:

- a) El título dice qué, dónde y cuándo: "Ozono troposférico: media mensual, enero a abril de 2024"
- b) "Valores" como rótulo del eje y alcanza
- c) La fuente de los datos se indica en el gráfico
- d) La unidad ($\mu\text{g}/\text{m}^3$) va en el rótulo del eje

66. Encapsular el gráfico en una función:

- a) Se escribe una vez y se llama varias
- b) El color puede pasar como argumento
- c) Hace el gráfico más lento
- d) Evita que dos copias del mismo bloque se desincronicen

67. Guardar el gráfico:

- a) `fig.savefig("g.png", dpi=300)` guarda la figura en un archivo
- b) `ggsave("g.png")` guarda el último gráfico
- c) Las figuras van a salidas/ y no se versionan: se regeneran
- d) La forma correcta es hacer captura de pantalla

68. Para graficar una serie en el tiempo con matplotlib:

- a) La columna fecha tiene que ser datetime (parse_dates al leer, o pd.to_datetime()), no texto
- b) Si la fecha queda como texto, matplotlib la trata como categorías y el eje no respeta las distancias entre fechas
- c) ax.plot(df["fecha"], df["o3"]) acepta directamente una columna datetime
- d) Hay que convertir la fecha a número a mano antes de graficar

69. matplotlib.dates:

- a) Se carga con import matplotlib.dates as mdates
- b) ax.xaxis.set_major_formatter(mdates.DateFormatter("%d/%m")) cambia cómo se escriben las etiquetas del eje
- c) DateFormatter modifica los datos de la columna fecha
- d) "%a %d" muestra el día de la semana abreviado y el número de día

70. Códigos de formato de fecha:

- a) %d día, %m mes, %Y año con cuatro cifras
- b) %H hora, %M minuto
- c) %Y y %y son lo mismo
- d) Los mismos códigos sirven en format() de R y en .dt.strftime() de pandas

71. Antes de graficar una serie:

- a) Con datos minutales, graficar todo produce una mancha; agregar por hora o por día es una decisión
- b) pd.Grouper(key="fecha", freq="D") agrega por día
- c) Después de groupby() hace falta reset_index() para tener la fecha como columna
- d) ax.plot() ordena las fechas solo: no importa que los datos vengan desordenados

Clase 9 — Unión de fuentes y gráficos por grupo

72. inner_join() / merge(how="inner"):

- a) Conserva solo las filas cuya clave existe en ambos lados
- b) Para O3 y PM2.5 la clave fue estación + fecha
- c) Las filas que quedan fuera se borran del disco
- d) Lo que queda fuera se puede graficar, pero no comparar

73. Color:

- a) geom_line(color = "blue") es un color fijo
- b) aes(color = estacion) mapea la variable a color
- c) El color fijo produce leyenda
- d) Con aes(), ggplot parte los datos por grupo sin bucle

74. Paneles y apilado:

- a) facet_wrap(~ estacion) da un panel por estación
- b) bind_rows() equivale a pd.concat()

- c) `pd.merge()` apila tablas
- d) En `patchwork`, / apila gráficos

75. Perfil horario, boxplot y dispersión:

- a) Agrupar por hora del día (0-23) no es lo mismo que por hora del calendario
 - b) El boxplot muestra mediana, cuartiles y valores extremos
 - c) El boxplot muestra solo la media
 - d) La pared vertical en $PM_{2.5} \approx 3$ es el piso del sensor
-

Clase 10 — Series de tiempo: una primera mirada

76. Grilla de tiempo regular:

- a) Observaciones a intervalos iguales
- b) Un paso corto conserva la variabilidad
- c) Remuestrear a paso horario es siempre mejor
- d) `df.resample("10min").mean()` remuestrea con índice `datetime`

77. Rezago y autocorrelación:

- a) `x.shift(1) / lag(x)` desplaza la serie un paso
- b) En el gráfico de rezago, puntos sobre la diagonal indican autocorrelación positiva
- c) La autocorrelación es la correlación con otra serie
- d) La ACF calcula la autocorrelación para varios rezagos k

78. Correlación cruzada:

- a) Mide la correlación entre dos series a distintos desfases
 - b) Se calculó entre O_3 y $PM_{2.5}$ en Curva de Maroñas
 - c) Una correlación alta implica causalidad
 - d) El rezago donde la correlación es máxima sugiere si una serie va adelantada
-

Lecturas obligatorias

79. Tukey (1962), *The Future of Data Analysis*:

- a) El análisis de datos es una ciencia empírica más amplia que la estadística matemática
- b) Es mejor una respuesta aproximada a la pregunta correcta que una exacta a la pregunta equivocada
- c) La exploración de los datos es parte del análisis, no un paso previo
- d) El análisis debe limitarse a probar hipótesis fijadas de antemano

80. Breiman (2001), *Statistical Modeling: The Two Cultures*:

- a) Una cultura supone un modelo estocástico del mecanismo que genera los datos
- b) La otra trata al mecanismo como caja negra y evalúa por capacidad predictiva
- c) La caja negra de la clase 2 es la de este artículo
- d) Los comentaristas (Cox, Efron, Hoadley, Parzen) son coautores del artículo

81. Hernán, Hsu y Healy (2019), *A Second Chance to Get Causal Inference Right*:

- a) Clasifican las tareas de la ciencia de datos en descripción, predicción e inferencia causal
- b) La inferencia causal requiere supuestos y conocimiento del dominio que los datos por sí solos no dan
- c) Predecir bien alcanza para saber qué pasa si se interviene
- d) Sostienen que la ciencia de datos debe reconocer la inferencia causal como tarea distinta

82. Bueno de Mesquita y Fowler (2021), *Thinking Clearly with Data*:

- a) Correlación no implica causalidad: hay que pensar en confusores y causalidad inversa
- b) Sin variación en la variable explicativa no hay correlación que estimar
- c) Seleccionar los casos según la variable de resultado sesga la conclusión
- d) Una correlación fuerte alcanza para afirmar un efecto causal

83. McKinney (2022), *Python for Data Analysis*:

- a) Es el manual de pandas, escrito por su creador
- b) DataFrame y Series son sus estructuras centrales
- c) Cubre lectura de archivos, limpieza, groupby y series de tiempo
- d) Es un libro sobre R

84. VanderPlas (2023), *Python Data Science Handbook*:

- a) Cubre NumPy, pandas, Matplotlib y scikit-learn
- b) Sus capítulos de Matplotlib son la referencia para los gráficos del curso
- c) La parte de scikit-learn queda fuera del alcance de esta materia
- d) Es un manual de SQL

85. Wickham, Çetinkaya-Rundel y Grolemund (2023), *R for Data Science*:

- a) El flujo es importar → ordenar → transformar → visualizar → modelar → comunicar
- b) Datos ordenados (*tidy*): cada variable una columna, cada observación una fila, cada valor una celda
- c) Usa el tidyverse: ggplot2, dplyr, readr
- d) Está escrito para Python

86. Healy (2018), *Data Visualization: A Practical Introduction*:

- a) La posición sobre un eje común se lee con más precisión que el área o el color
- b) Un gráfico puede engañar por la escala de los ejes sin que haya datos falsos
- c) Los ejemplos están hechos con ggplot2
- d) Recomienda gráficos de torta y en 3D

87. Knafllic (2015), *Storytelling with Data*:

- a) Un gráfico sostiene una afirmación, no “muestra los datos”
- b) Hay que quitar lo que no aporta: bordes, grillas, colores innecesarios
- c) El título dice el mensaje, no el tema
- d) Cuantos más colores, mejor

Clave de corrección

Para cada pregunta se indica el valor de verdad de cada opción y se desarrolla la fundamentación. Las explicaciones están escritas para que puedan leerse de forma independiente: quien no haya resuelto la pregunta debería poder comprender el tema a partir de la respuesta.

Clase 1 — Git y trabajo reproducible

1. a V · b V · c V · d F.

La carpeta del enunciado ilustra el modo en que habitualmente se intenta conservar el historial de un trabajo sin una herramienta adecuada: se guardan copias sucesivas del archivo con nombres cada vez más largos. Ese procedimiento produce muchos archivos y ninguna respuesta a las tres preguntas que realmente importan cuando se trabaja con datos: cuál de las versiones es la correcta, qué se modificó entre una versión y la siguiente, y quién realizó cada modificación y por qué motivo.

Un sistema de control de versiones resuelve exactamente ese problema. Git mantiene el historial completo de una carpeta: cada versión guardada queda registrada de forma permanente con el nombre de su autor, la fecha y hora, y un mensaje que explica el cambio. Es posible comparar dos versiones cualesquiera línea por línea y volver a cualquier estado anterior. Por ese motivo las opciones a, b y c son verdaderas: son, precisamente, las tres capacidades que definen la herramienta.

La opción d es falsa y describe lo contrario de lo que Git hace. El control de versiones no impide el trabajo simultáneo; está diseñado para permitirlo. Varias personas pueden modificar la misma carpeta en paralelo, cada una en su propia copia, y luego integrar sus cambios. Cuando dos personas modifican partes distintas de un archivo, la integración es automática; cuando modifican las mismas líneas, la herramienta lo señala como conflicto y pide una decisión, pero en ningún caso bloquea el trabajo. En este curso, Git no es una herramienta accesoria sino el método de trabajo: todo lo que produce un equipo pasa por el repositorio.

2. a V · b V · c F · d V.

Una confirmación (*commit*) es la unidad básica del historial de Git. Puede pensarse como una fotografía del estado de todos los archivos del proyecto en un instante determinado. Esa fotografía no se guarda sola: se acompaña de tres datos que la identifican. El autor, es decir, la persona que realizó el trabajo, tomado de la configuración de su cuenta; la fecha y hora en que se confirmó; y el mensaje, escrito por el autor, que explica el motivo del cambio. Además, Git almacena la diferencia exacta respecto de la confirmación anterior, línea por línea, de modo que siempre es posible saber qué se agregó, qué se eliminó y qué se modificó. Por todo ello las opciones a y b son verdaderas.

La opción d también es verdadera y es una de las razones principales para usar la herramienta: cualquier confirmación del historial puede recuperarse. Si una limpieza de datos resulta equivocada, si un gráfico dejó de generarse tras un cambio, si se quiere ver cómo estaba el análisis hace tres semanas, basta con volver a la confirmación correspondiente.

La opción c es falsa. Las confirmaciones se almacenan en el disco de la máquina, dentro de una carpeta oculta llamada `.git` que se encuentra en la raíz del proyecto. Apagar la computadora, cerrar el editor o cerrar sesión no las afecta. Lo único que puede perderlas es borrar esa carpeta o borrar el proyecto completo, y aun en ese caso, si el trabajo fue sincronizado con GitHub, existe una copia remota desde la cual recuperarlo.

3. a V · b F · c F · d V.

Esta pregunta distingue tres operaciones que los estudiantes suelen confundir porque las tres se describen coloquialmente como “guardar”. La primera es guardar el archivo en el editor, con Ctrl+S o el botón correspondiente. Esa operación escribe el contenido en el disco de la máquina local y nada más: el archivo modificado existe en la carpeta del proyecto, pero Git todavía no lo ha registrado. Por eso la opción a es verdadera.

La segunda operación es confirmar. Consiste en indicarle a Git que el estado actual del archivo, o de un conjunto de archivos, debe incorporarse al historial como una nueva confirmación, con su autor, fecha y mensaje. Mientras esa operación no se realice, el cambio no forma parte del historial, y la opción b es falsa. En GitHub Desktop, confirmar equivale a tildar los archivos y presionar *Commit*; en el panel de VS Code, a preparar el archivo con el signo +, escribir el mensaje y presionar *Confirmar*.

La tercera operación es publicar o sincronizar. Confirmar registra el cambio en el historial local, pero ese historial vive en la máquina de Ana. Para que el equipo lo vea en GitHub es necesario enviarlo al repositorio remoto, lo que en GitHub Desktop se hace con *Push origin* y en VS Code con *Sincronizar*. Como Ana no realizó esa operación, la opción c es falsa.

La opción d resume la situación: para que el cambio quede registrado y sea visible, faltan las dos operaciones posteriores al guardado. Guardar, confirmar y publicar son pasos distintos y sucesivos.

4. a F · b V · c V · d F.

El mensaje de confirmación es la parte del registro que escribe la persona, y su función es documentar el motivo del cambio. Git registra automáticamente qué se modificó, quién lo hizo y cuándo; lo que no puede registrar por sí solo es por qué. Ese porqué es lo que permite que un tercero, o el propio autor semanas después, lea el historial y reconstruya las decisiones tomadas sin necesidad de abrir el código. Por ese motivo la opción c es verdadera.

La opción a es falsa. Un mensaje como “cambios” es técnicamente válido, pero no aporta información: todo commit contiene cambios, y el historial resultante es una lista de entradas indistinguibles entre sí. La opción d también es falsa: un mensaje vacío tiene el mismo defecto, y la mayoría de las interfaces no permiten confirmar sin escribir uno.

La opción b muestra el formato que se espera en el curso: una etiqueta que identifica la etapa del trabajo (“limpieza”, “obtención”, “informe”, “estructura”), seguida de dos puntos y una descripción concreta de la acción realizada. Con mensajes de ese tipo, el historial leído de abajo hacia arriba cuenta la historia del análisis: se obtuvieron los

datos del INE, se trataron los precios en cero como faltantes, se agregó el intervalo de confianza al pronóstico. Un historial así es, en sí mismo, documentación del proyecto.

5. a V · b F · c V · d F.

La regla del curso es que se versiona lo que no se puede regenerar, y no se versiona lo que sí se puede regenerar a partir de lo versionado. El código en cualquiera de sus formas (.py, .R, .qmd) se versiona, porque es el producto del trabajo. El script que obtiene los datos se versiona por la misma razón: es código, y además es lo que permite que cualquier persona reconstruya el conjunto de datos desde la fuente. Por ello las opciones a y c son verdaderas.

Los datos crudos no se versionan. Hay dos motivos. El primero es práctico: los archivos de datos suelen ser grandes, no cambian línea por línea como el código, y hacen que el repositorio crezca hasta volverse inmanejable. El segundo es metodológico: si el script de obtención está versionado, los datos se regeneran ejecutándolo, y versionarlos además es redundante. Los datos crudos se guardan en datos/crudo/, carpeta que el archivo .gitignore excluye del control de versiones. La opción b es falsa.

Las contraseñas, claves de API y credenciales no deben incorporarse nunca al repositorio, bajo ninguna forma. El historial de Git es permanente: una clave confirmada por error permanece en el historial aunque se elimine del archivo en una confirmación posterior, y si el repositorio es público, queda expuesta. Lo mismo vale para los datos personales. La opción d es falsa.

6. a F · b V · c V · d V.

Un conflicto de fusión se produce cuando dos personas modificaron las mismas líneas del mismo archivo, cada una en su copia, y luego intentan integrar los cambios. Git puede combinar automáticamente modificaciones en partes distintas de un archivo, pero cuando las modificaciones se superponen no tiene forma de decidir cuál es la correcta. En lugar de elegir, marca el conflicto y presenta ambas versiones. La opción b es verdadera.

La opción a es falsa. Un conflicto no indica que el repositorio esté dañado, y clonarlo de nuevo no lo resuelve: el conflicto reaparecería en la siguiente sincronización. Es una situación prevista por la herramienta y forma parte del trabajo normal en equipo.

La resolución consiste en abrir el archivo, examinar las dos versiones que la interfaz muestra, decidir cuál se conserva (o combinar ambas), y confirmar el resultado. La opción c es verdadera.

La opción d señala algo específico de este curso. Cuando dos integrantes escribieron versiones distintas de las mismas líneas de decisiones.md, lo que está en juego no es un detalle técnico sino una decisión de método que el equipo tenía pendiente: por ejemplo, si un cliente activo se define por 60 o por 90 días sin compras. Corresponde conversarlo, elegir con fundamento y dejar la decisión registrada en el propio archivo decisiones.md, junto con la alternativa descartada.

7. a V · b V · c F · d V.

El curso organiza el trabajo en dos tipos de repositorio. Cada equipo tiene un repositorio compartido, con la estructura de proyecto y los archivos de trabajo

(bitacora.md, decisiones.md, datos/, salidas/). Además, cada estudiante mantiene una bitácora personal en un repositorio propio, con una confirmación semanal. La opción a es verdadera.

La política de autoría establece que cada estudiante confirma su propio trabajo desde su propia cuenta. El motivo es que el historial de confirmaciones es la evidencia individual de participación: muestra quién trabajó, cuándo y sobre qué. Por eso la opción b es verdadera y la c es falsa. Si una sola persona sube el trabajo de todos, el historial no permite distinguir la contribución de cada integrante, y la regla del curso es que en ese caso la participación se evalúa como mínima para todo el equipo. El integrador del equipo fusiona el trabajo de los demás, pero no es el autor de todo.

La opción d es verdadera y describe una diferencia importante respecto de la entrega tradicional. Como el repositorio del equipo está en GitHub, el docente puede ver el trabajo a medida que ocurre: qué se intentó cada semana, qué falló, cómo se resolvió. La evaluación no se apoya únicamente en el producto final sino en el proceso, y el historial de commits es parte de ella. Un historial que muestra trabajo distribuido en el tiempo y entre los integrantes es lo que se espera.

8. a V · b F · c V · d F.

El archivo bitacora.md es el registro semanal del equipo. Cada entrada tiene un máximo de 150 palabras y tres campos: qué intentamos, qué falló y qué decidimos. La opción a es verdadera.

La opción b es falsa y apunta a la regla más importante de la bitácora: el segundo campo es obligatorio. Una semana sin nada que haya fallado no se interpreta como una semana exitosa sino como una semana sin trabajo, porque en el trabajo con datos reales siempre aparecen dificultades: un archivo con codificación incorrecta, una columna con valores inesperados, una unión que pierde filas, un script que funciona en una máquina y no en otra. Registrar esas dificultades es lo que hace útil la bitácora, tanto para el propio equipo como para la evaluación.

El archivo decisiones.md cumple otra función. Registra cada decisión de limpieza o de método, junto con la alternativa que se descartó y el motivo. Por ejemplo: se decidió tratar los precios en cero como valores faltantes, en lugar de mantenerlos, porque en el sistema de origen el cero indica que el campo no fue completado. La opción c es verdadera.

La opción d es falsa. En la carpeta datos/ se versiona el script que obtiene los datos, no los datos crudos, por las razones expuestas en la pregunta 5. Quien clona el repositorio ejecuta el script y obtiene los mismos datos.

Clase 2 — El proceso generador de datos

9. a F · b V · c V · d F.

En la práctica profesional suele recibirse un archivo .csv o una tabla de una base de datos, y es natural asumir que ese archivo es la realidad. No lo es. Los datos no existen en la naturaleza: se construyen mediante un acto de medición. Detrás de todo conjunto de datos hay un mecanismo —físico, biológico, social o económico— que produce los eventos que se intenta registrar. Ese mecanismo es el proceso generador de datos

(DGP, por su sigla en inglés), y el archivo recibido es solo el resultado de observarlo a través de un instrumento, con las limitaciones de ese instrumento. Por eso la opción a es falsa y la b es verdadera.

Breiman (2001) propuso pensar ese mecanismo como una caja negra: un sistema cuyo interior no es accesible, del que solo se observan las entradas y las salidas. Dentro de la caja ocurren interacciones complejas, hay ruido e incertidumbre. La dinámica de la clase 2, en la que cada estudiante tocó un objeto sin verlo y registró textura, rigidez, aristas y peso estimado, fue una puesta en escena de esa idea. La opción c es verdadera.

La opción d es falsa y es el punto central. Un modelo estadístico o un algoritmo de aprendizaje automático procesa únicamente matrices numéricas. No sabe si el peso se midió con una balanza o se estimó al tacto, no sabe si hubo sesgo en quién decidió responder, no comprende la relación causal subyacente. El modelo no conoce el DGP; solo ajusta una función a la muestra que se le entrega. Todo lo que el investigador sabe sobre cómo se generaron los datos tiene que aportarlo él, porque el modelo no lo va a descubrir.

10. a V · b V · c F · d V.

Las variables registradas en la dinámica de la caja negra sirven para repasar la tipología básica de variables, que reaparece a lo largo de todo el curso.

La textura, con valores suave, rugoso y poroso, es una variable nominal: sus categorías son etiquetas sin ningún orden entre ellas. No tiene sentido decir que rugoso es mayor que suave. La opción a es verdadera.

La rigidez, con valores blando, medio y rígido, es una variable ordinal: sus categorías tienen un orden natural, pero no hay una distancia numérica definida entre ellas. Se sabe que rígido es más que medio, pero no cuánto más. La opción b es verdadera.

La cantidad de aristas es un conteo: toma valores enteros no negativos (0, 4, 6, 8, 12). Es una variable numérica discreta. La opción c la clasifica como continua y es falsa. La distinción importa porque un conteo no puede tomar valores intermedios —no hay 4,5 aristas— y eso condiciona qué distribuciones y qué gráficos son apropiados.

El peso estimado en gramos puede tomar cualquier valor real positivo, con la precisión que permita el instrumento. Es una variable numérica continua. La opción d es verdadera.

En el código de la clase, esta tipología se tradujo en tipos de dato: la textura se convirtió a `category`, la rigidez a `Categorical` con `ordered=True` y el orden explícito de las categorías, y las dos numéricas quedaron como enteros y decimales.

11. a V · b V · c F · d V.

La clase 2 presentó dos formas de generar datos sintéticos a partir de una muestra pequeña.

El *bootstrap* (Efron, 1979) parte de un supuesto: la muestra observada es la mejor aproximación disponible de la población desconocida. Bajo ese supuesto, se generan nuevas muestras extrayendo observaciones de la muestra original con reemplazo, es decir, devolviendo cada observación extraída antes de extraer la siguiente. Un mismo

objeto puede aparecer varias veces en la nueva muestra y otro puede no aparecer. En el código, `df.sample(n=100, replace=True)` produce cien filas a partir de las diez originales. La opción a es verdadera.

La generación paramétrica supone una distribución —en el ejemplo, la normal—, estima sus parámetros a partir de la muestra (media y desviación estándar del peso) y genera nuevas observaciones a partir de esa distribución con `np.random.normal()`. Como ambos procedimientos involucran números aleatorios, el resultado cambia en cada ejecución salvo que se fije una semilla. `np.random.seed(42)` fija el punto de partida del generador, de modo que la secuencia aleatoria sea la misma en cada corrida y el resultado sea reproducible. La opción b es verdadera.

La opción c es falsa. Suponer una distribución normal equivale a suponer que la caja contiene un solo tipo de objeto, con pesos que varían alrededor de un valor central. Si la caja contiene objetos de naturaleza distinta —un tornillo, una piedra, una pelota—, la distribución real del peso tiene varios modos, y la normal ajustada genera valores intermedios que no corresponden a ningún objeto existente. Son datos “fantasma”: estadísticamente plausibles bajo el modelo, imposibles bajo el mecanismo real.

La opción d es verdadera con esa salvedad. Cuando el modelo generador es adecuado, los datos sintéticos permiten compartir información para investigar sin exponer los datos originales ni comprometer la privacidad de las personas.

12. a F · b V · c F · d V.

La expresión *garbage in, garbage out* resume una propiedad de todo procedimiento de análisis: la calidad del resultado está acotada por la calidad de los datos de entrada.

La opción a es falsa. Un modelo no corrige los errores de medición ni los sesgos de selección; no dispone de ninguna información que le permita distinguir un valor bien medido de uno mal medido. Lo que hace es ajustarse a los datos que recibe, con sus errores incluidos. Si la balanza estaba descalibrada, el modelo aprende la escala descalibrada. Si solo respondieron los clientes satisfechos, el modelo aprende que todos los clientes están satisfechos. Y como los métodos de ajuste buscan reproducir los patrones de la muestra con la mayor fidelidad posible, los sesgos no solo se conservan sino que tienden a amplificarse. La opción b es verdadera.

La opción c es falsa. Generar datos sintéticos a partir de una muestra pobre produce más datos igualmente pobres: el *bootstrap* remuestrea lo que hay, y la generación paramétrica reproduce la distribución estimada de lo que hay. Ninguno de los dos procedimientos agrega información que no estuviera en la muestra original. Lo mismo vale para la complejidad del algoritmo: un modelo más sofisticado no compensa un conjunto de datos con baja diversidad o etiquetas ruidosas.

La opción d expresa la conclusión de la clase. El límite de lo que un modelo puede aprender no lo define la complejidad del algoritmo sino la riqueza estadística y la fidelidad del proceso generador de datos que se utilizó para construir la muestra. Un buen conjunto de datos tiene alta variación entre clases (cubre el dominio), alta variación dentro de cada clase (es robusto al entorno) y mínimo ruido en las etiquetas. Ninguna de esas tres propiedades se obtiene después, en la etapa de modelado.

Clase 3 — Objetos, tipos y errores

13. a $V \cdot b$ $V \cdot c$ $F \cdot d$ V .

Vectorizar significa aplicar una operación a todos los elementos de una estructura de una sola vez, sin escribir un bucle que recorra los elementos uno por uno. Es una idea central para el trabajo con datos, porque las tablas tienen miles o millones de filas y las operaciones sobre columnas completas son el caso habitual. La opción a es verdadera.

En R, la vectorización es el comportamiento por defecto. El vector es la estructura básica del lenguaje y todas las operaciones aritméticas, lógicas y muchas funciones actúan elemento a elemento. La expresión `c(1, 2, 3) * 10` devuelve `10 20 30`; `c(1, 2, 3) > 1` devuelve `FALSE TRUE TRUE`; `sqrt(c(4, 9, 16))` devuelve `2 3 4`. No hace falta indicar nada especial. La opción b es verdadera.

En Python, la situación es distinta y es fuente frecuente de errores en las primeras semanas. La lista, `[1, 2, 3]`, no vectoriza. El operador `*` aplicado a una lista y un entero no multiplica los elementos: repite la lista. `[1, 2, 3] * 10` devuelve una lista de treinta elementos, `[1, 2, 3, 1, 2, 3, ...]`. Del mismo modo, `[1, 2, 3] + [4]` no suma: concatena y devuelve `[1, 2, 3, 4]`. La opción c es falsa.

Para obtener en Python el comportamiento vectorizado de R hay que usar otra estructura: el arreglo de NumPy. `np.array([1, 2, 3]) * 10` devuelve `array([10, 20, 30])`. Las columnas de un DataFrame de pandas están construidas sobre arreglos de NumPy, por lo que `df["peso"] * 1000` sí multiplica cada valor de la columna. La opción d es verdadera. En la práctica, en Python se trabaja casi siempre con pandas y NumPy, y las listas quedan para colecciones pequeñas de valores heterogéneos.

14. a $V \cdot b$ $F \cdot c$ $V \cdot d$ V .

Un vector es una secuencia ordenada de elementos del mismo tipo. Las dos propiedades importan: ordenada, porque cada elemento tiene una posición y se accede a él por esa posición; del mismo tipo, porque todos los elementos son números, o todos son texto, o todos son lógicos, y esa homogeneidad es lo que permite vectorizar las operaciones. La opción c es verdadera.

En R, el vector se construye con la función `c()`, cuyo nombre viene de *combine*: `c(1, 2, 3)` combina tres valores en un vector numérico de longitud tres. La opción a es verdadera.

La opción b es falsa. En Python, `[]` construye una lista, no un vector. La lista es también una secuencia ordenada, pero no exige que sus elementos sean del mismo tipo: `[1, "a", True]` es una lista válida que conserva un entero, un texto y un lógico sin convertir ninguno. Y, como se vio en la pregunta anterior, la lista no vectoriza las operaciones aritméticas. Por ambas razones, la lista de Python no se comporta como el vector de R. El equivalente real del vector de R en Python es el arreglo de NumPy.

La opción d es verdadera y suele sorprender. En R no existe un tipo escalar separado: no hay “números sueltos”. El valor 1 escrito solo es un vector numérico de longitud uno. Puede comprobarse con `length(1)`, que devuelve 1, y con `is.vector(1)`, que devuelve `TRUE`. Esta uniformidad es lo que hace que las operaciones se apliquen del mismo modo a un valor que a mil: en ambos casos se está operando sobre un vector.

15. a F · b V · c V · d V.

La asignación es la operación que guarda un valor bajo un nombre. En Python se escribe `edad = 35`; en R, `edad <- 35`. La lectura correcta de ambas expresiones es “guardá el valor 35 bajo el nombre edad”. La opción c es verdadera: los dos operadores cumplen la misma función.

La opción a es falsa. El signo `=` en la asignación no representa una igualdad matemática. No se está afirmando que `edad` y `35` sean iguales; se está creando un nombre y asociándolo a un valor. La diferencia se vuelve evidente con la expresión de la opción b, `edad = edad + 1`, que como ecuación no tiene solución y como asignación es perfectamente válida: toma el valor que hay guardado bajo `edad`, le suma uno, y guarda el resultado bajo el mismo nombre. La opción b es verdadera.

La opción d es verdadera y tiene una consecuencia práctica que conviene tener presente. El nombre y el valor son cosas distintas. Cuando se ejecuta `peso = 45.0` y luego `peso = 120.5`, el nombre `peso` pasa a apuntar al nuevo valor y el anterior deja de estar disponible. Lo mismo ocurre con estructuras grandes: si una celda de un notebook contiene `df = df.dropna()` y se ejecuta dos veces, la segunda ejecución opera sobre el resultado de la primera, no sobre el data frame original. El nombre es el mismo; el contenido, no. Por eso, cuando un análisis parece dar resultados distintos de los esperados, una de las primeras cosas que conviene verificar es si alguna celda se ejecutó más de una vez.

16. a F · b V · c V · d V.

Los tipos elementales son las categorías básicas de valores que un lenguaje distingue. Los cuatro que importan al inicio son el entero (`int` en Python, `integer` en R), el decimal (`float` en Python, `numeric` o `double` en R), el texto (`str` en Python, `character` en R) y el lógico (`bool` en Python, `logical` en R). A ellos se suma la ausencia de valor: `None` y `NaN` en Python, `NA` en R. La opción c y la opción d son verdaderas.

La opción a es falsa y señala el error más frecuente de las primeras prácticas. Las comillas definen el tipo. `45.0` sin comillas es un número decimal; `"45.0"` con comillas es una cadena de texto formada por los caracteres 4, 5, punto y 0. Se ven iguales en pantalla, pero son objetos de tipo distinto y se comportan de forma distinta.

La opción b muestra la consecuencia. El operador `+` aplicado a dos números suma: `45.0 + 1` devuelve `46.0`. El mismo operador aplicado a dos textos concatena: `"45.0" + "1"` devuelve `'45.01'`. No hay error, y ese es el problema: la operación se ejecuta y produce un resultado sin sentido. En la práctica esto ocurre cuando un archivo CSV se lee con un separador decimal incorrecto y los pesos quedan como texto. Todas las operaciones posteriores siguen funcionando, pero los promedios, las sumas y los gráficos son erróneos. Verificar el tipo de cada columna con `df.dtypes` o `str(df)` inmediatamente después de leer un archivo es la forma de detectarlo.

17. a F · b V · c V · d V.

Los mensajes de error no son un castigo sino información. La clase 3 identificó los tres errores más frecuentes del primer día y lo que cada uno significa.

`NameError: name 'df' is not defined` en Python, o `Error: object 'df' not found` en R, indica que el nombre `df` no existe en la sesión. No significa que el código esté mal

escrito ni que falte un archivo: significa que la instrucción que creaba `df` no se ejecutó. En un notebook, esto ocurre casi siempre porque se corrió la celda 5 antes que la celda 2. Es un error de orden de ejecución. La opción a lo confunde con un error de archivo y es falsa.

`TypeError` en Python, o `non-numeric argument to binary operator` en R, indica que la operación no corresponde al tipo del objeto: se intentó sumar un texto con un número, o calcular la media de una columna de texto. Es un error de tipo. La opción b es verdadera.

`FileNotFoundError` en Python, o `cannot open file` en R, indica que el archivo no está donde se lo buscó. Es un error de ubicación, casi siempre causado por una ruta incorrecta o un directorio de trabajo distinto del esperado. La opción c es verdadera.

La opción d describe cómo leer un mensaje de error de Python. El mensaje se lee de abajo hacia arriba: la última línea dice qué ocurrió (el tipo de error y una descripción breve); las líneas anteriores dicen dónde ocurrió (el archivo, la línea, la función). Ninguno de los tres errores se resuelve reescribiendo la línea al azar; cada uno tiene una causa identificable y una solución distinta.

18. a F · b V · c V · d V.

Esta pregunta examina una propiedad del vector de R que tiene consecuencias importantes en la carga de datos.

Un vector exige que todos sus elementos sean del mismo tipo. Cuando se intenta construir uno con valores de tipos distintos, como en `c(1, "a", TRUE)`, R no produce un error. Lo que hace es convertir todos los elementos al único tipo capaz de representarlos a todos. Un número puede escribirse como texto, un lógico puede escribirse como texto, pero un texto arbitrario no puede convertirse en número ni en lógico. Por lo tanto, el tipo común es el texto, y el resultado es el vector "1" "a" "TRUE", de longitud tres. La opción a es falsa; las opciones b y d son verdaderas.

La opción c es verdadera y es el aspecto que conviene retener. La conversión es silenciosa: no aparece ningún mensaje, ninguna advertencia. El código sigue ejecutándose con un vector de texto donde el programador creía tener números. La consecuencia práctica aparece al leer archivos. Una columna numérica que contiene un solo valor de texto —un espacio, un guion, la palabra “s/d” para indicar un faltante— se convierte entera en texto. A partir de ese momento, `mean()` da error, las comparaciones numéricas no funcionan y los gráficos tratan los valores como categorías. La forma de detectarlo es revisar los tipos con `str()` después de leer, y la forma de corregirlo es identificar el valor problemático, tratarlo como faltante y convertir la columna con `as.numeric()`.

19. a V · b V · c F · d F.

Un data frame es la estructura de datos central del curso, tanto en R como en Python (donde se llama `DataFrame`). Conviene entender cómo está construido.

La clase 3 lo presentó a partir del diccionario de Python. Un diccionario asocia nombres a valores: `{"textura": ["suave", "rugoso"], "peso": [45.0, 120.5]}`. Cada clave es el nombre de una columna y cada valor es la columna entera. Un data frame es exactamente eso, con una exigencia adicional: todas las columnas deben tener el

mismo largo. Por eso la opción a es verdadera —un data frame es una colección de columnas— y la opción d es falsa: no se admiten columnas de distinto largo, porque cada fila debe tener un valor en cada columna.

Cada columna tiene su propio tipo. La columna `textura` es de texto; la columna `peso` es numérica; puede haber una columna lógica y otra de fechas. Dentro de una columna, todos los valores son del mismo tipo, como en un vector. La opción b es verdadera.

Una fila, en cambio, mezcla tipos: contiene un texto, un número, un lógico. No tiene un tipo único. La opción c es falsa. Esta asimetría explica por qué se trabaja por columnas y no por filas: las operaciones vectorizadas, los resúmenes estadísticos y los gráficos actúan sobre columnas, que son homogéneas. Cuando se necesita seleccionar filas, se hace mediante condiciones sobre columnas, como se ve en la pregunta siguiente.

20. a F · b V · c V · d V.

La expresión `df["peso"] > 50` compara cada valor de la columna `peso` con el número 50. Como la columna es un vector, la comparación se vectoriza: no devuelve un único resultado sino uno por cada fila. El resultado es una secuencia de valores lógicos, `False`, `True`, `False`, ..., del mismo largo que la columna. Esa secuencia se denomina máscara booleana. La opción a es falsa y la b es verdadera.

La máscara tiene dos usos principales. El primero es filtrar. `df[df["peso"] > 50]` devuelve un nuevo data frame con únicamente las filas en que la máscara es `True`, es decir, las filas cuyo peso supera 50. En R, la misma operación se escribe `df[df$peso > 50, ]` o, con `dplyr`, `filter(df, peso > 50)`. La opción c es verdadera.

El segundo uso es contar. Los valores lógicos se comportan como números cuando se los suma: `True` vale uno y `False` vale cero. Por lo tanto, `(df["peso"] > 50).sum()` devuelve la cantidad de filas que cumplen la condición. En R, `sum(df$peso > 50)` hace lo mismo. La opción d es verdadera. Del mismo modo, `(df["peso"] > 50).mean()` devuelve la proporción de filas que cumplen la condición, un recurso que se usa en la clase 7 para calcular la proporción de valores faltantes por día.

21. a V · b V · c F · d F.

Función y método son dos formas de aplicar una operación a un objeto, y la diferencia explica por qué el código de Python está lleno de puntos y el de R no.

Una función se aplica al objeto, que se le pasa como argumento: `len(x)` en Python, `length(x)` en R. La función existe con independencia del objeto. Las opciones a y d se refieren a esto; la a es verdadera y la d es falsa, porque `length()` es una función, no un método.

Un método pertenece al objeto y se invoca con la notación de punto: `df.head()`, `df.dropna()`, `"texto".upper()`. El método está definido por el tipo del objeto: un `DataFrame` tiene el método `head()`, una cadena de texto tiene el método `upper()`, y ninguno de los dos tiene el método del otro. La opción b es verdadera.

Python usa métodos de forma generalizada; casi toda la manipulación de datos con `pandas` se hace encadenando métodos: `df.dropna().groupby("mes").mean()`. R usa muy pocos métodos con esta notación; sus operaciones son mayoritariamente funciones, y el encadenamiento se hace con el operador `%>%` o `|>`: `df |> na.omit() |> group_by(mes)`

|> summarise(media = mean(o3)). La opción c es falsa. Reconocer esta diferencia ayuda a leer código en ambos lenguajes y a buscar ayuda: en Python se busca “método head de DataFrame”; en R, “función head”.

22. a F · b F · c V · d V.

Una llamada como `pd.read_csv("datos.csv", sep=";", decimal=",", encoding="latin-1")` contiene un dato y varias instrucciones. El dato es el primer argumento: el nombre del archivo que se quiere leer. Los argumentos con nombre que siguen no son datos adicionales que se lean del archivo; son instrucciones sobre cómo interpretarlo. `sep=";"` indica que las columnas están separadas por punto y coma; `decimal=","` indica que la coma es el separador decimal; `encoding="latin-1"` indica cómo interpretar los bytes del archivo. La opción d es verdadera y la a es falsa.

La opción b es falsa. Modificar `sep` no cambia el archivo en el disco. Lo que cambia es la lectura: la forma en que el programa divide cada línea en columnas. El archivo permanece intacto y puede leerse de nuevo con otros argumentos.

La opción c es la consecuencia y es verdadera. El mismo archivo leído con `sep=","` y con `sep=";"` produce dos tablas distintas. Si el archivo usa punto y coma y se lee con coma, el resultado es una tabla de una sola columna cuyo contenido es cada línea completa, con los puntos y coma incluidos. Si se lee con el separador correcto, se obtienen las columnas esperadas. La lectura de un archivo es una decisión del analista, y el archivo no tiene una única lectura posible. Por eso el perfilado comienza por identificar el delimitador, la codificación y el separador decimal antes de confiar en la tabla obtenida.

23. a V · b V · c F · d F.

R y Python numeran los elementos de una secuencia de forma distinta, y la diferencia produce errores silenciosos cuando se pasa de un lenguaje al otro.

R numera desde uno. En el vector `x <- c(10, 20, 30)`, el primer elemento es `x[1]` y vale 10; el tercero es `x[3]` y vale 30. La opción a es verdadera. La expresión `x[0]` no produce error: devuelve un vector vacío, `numeric(0)`, lo que suele pasar inadvertido hasta que un cálculo posterior falla. La opción d es falsa.

Python numera desde cero. En la lista `x = [10, 20, 30]`, el primer elemento es `x[0]` y vale 10; el tercero es `x[2]` y vale 30. La opción b es verdadera. La expresión `x[1]` devuelve el segundo elemento, 20, sin error alguno. La opción c es falsa.

La misma convención se aplica a las filas y columnas de una tabla. En el script de la clase 7, `ozono[1:10, 3]` en R selecciona las filas 1 a 10 y la tercera columna; el equivalente en pandas es `ozono.iloc[0:10, 2]`, donde `0:10` abarca las posiciones 0 a 9 (el límite superior queda excluido) y 2 es la tercera columna. Al traducir código entre los dos lenguajes, los índices son lo primero que hay que revisar.

Paquetes

24. a V · b F · c V · d V.

Un paquete (en Python también llamado biblioteca o módulo) es un conjunto de funciones desarrolladas por terceros que amplía las capacidades del lenguaje. El

lenguaje base de Python o de R incluye lo esencial —aritmética, estructuras de datos, control de flujo—, pero la lectura de archivos Excel, la construcción de gráficos, el manejo de fechas o la manipulación de tablas grandes se resuelven con paquetes. La opción a es verdadera.

La opción b es falsa. Los paquetes no forman parte de la instalación base. Deben instalarse en cada máquina, una vez, con una orden específica: `pip install pandas` en Python o `install.packages("pandas")` en R. Una vez instalado, el paquete queda disponible en el disco, pero no está activo en cada sesión: hay que cargarlo al inicio de cada script con `import pandas as pd` o `library(dplyr)`. Instalar es una operación única por máquina; cargar es una operación por sesión. La opción c es verdadera.

La opción d es verdadera. El archivo `requirements.txt` del repositorio del curso enumera los paquetes de Python que el proyecto necesita, uno por línea. Quien clona el repositorio ejecuta `pip install -r requirements.txt` y obtiene todos los paquetes con una sola orden, sin tener que descubrirlos uno por uno a medida que los scripts fallan. En R, el paquete `renv` cumple una función similar mediante el archivo `renv.lock`. Este mecanismo es parte de lo que hace reproducible un proyecto: no basta con compartir el código; hay que compartir también la lista de lo que el código necesita para ejecutarse.

25. a V · b V · c F · d V.

Esta pregunta insiste en la distinción entre instalar y cargar, que es fuente de dos errores opuestos.

Instalar se hace una vez por máquina. En Python, `pip install pandas` se ejecuta en la terminal (PowerShell en Windows, Terminal en Mac), no dentro del script. En R, `install.packages("ggplot2")` se ejecuta en la consola de RStudio. El paquete se descarga y queda guardado en el disco. Las opciones a y d son verdaderas.

Cargar se hace en cada sesión. `import pandas as pd` se escribe al principio de cada script de Python; `library(ggplot2)` al principio de cada script de R. Sin esa línea, las funciones del paquete no están disponibles, aunque el paquete esté instalado. La opción b es verdadera.

El primer error consiste en escribir el script sin la línea de carga, o ejecutar una celda que usa `pd` sin haber ejecutado antes la celda del `import`. El síntoma es `NameError: name 'pd' is not defined`. El segundo error es el de la opción c, que es falsa: incluir `install.packages()` en el script y ejecutarlo cada vez. Es innecesario, porque el paquete ya está instalado; demora el inicio del trabajo, porque descarga e instala de nuevo; y puede fallar si no hay conexión. Por eso en los scripts del curso las órdenes de instalación aparecen comentadas, como recordatorio, y solo se descomentan la primera vez.

26. a V · b F · c V · d V.

Los tres paquetes de Python que se usan a lo largo del curso son `numpy`, `pandas` y `matplotlib`. Son los que se instalan en la clase 2 con `pip install pandas numpy matplotlib` y los que figuran en `requirements.txt`. Las opciones a, c y d son verdaderas.

La opción b es falsa. `pynumpy` no es un paquete existente; es una transposición de las letras de `numpy`. La orden `pip install pynumpy` produce un mensaje del tipo `No matching`

distribution found, y la línea `import pynump` produce `ModuleNotFoundError`.

La pregunta tiene un propósito práctico. Los nombres de los paquetes hay que escribirlos exactamente, y los errores de tipeo son frecuentes: `pandas` y no `panda`, `matplotlib` y no `matplotlib.py`, `numpy` y no `numpy` con la `n` y la `m` intercambiadas. Cuando aparece un error de instalación o de importación, lo primero que conviene verificar es que el nombre esté bien escrito. Los nombres de los paquetes se distinguen entre mayúsculas y minúsculas y se escriben en minúsculas.

27. a `V · b` `V · c` `F · d` `V`.

Los paquetes de R que se usan en el curso son, principalmente, los del `tidyverse`. `ggplot2` produce gráficos; `dplyr` transforma tablas; `readr` lee archivos; `lubridate` gestiona fechas; `tidyr` reorganiza tablas entre formato ancho y largo. `tidyverse` es un paquete que agrupa a todos ellos, de modo que `library(tidyverse)` los carga de una vez. A ellos se suman `here` para las rutas, `readxl` para archivos Excel y `haven` para archivos de SPSS. Las opciones `a`, `b` y `d` son verdaderas.

La opción `c` es falsa. `pandas` es un paquete de Python, no de R. La orden `library(pandas)` en R produce el mensaje `Error in library(pandas) : there is no package called 'pandas'`, y `install.packages("pandas")` no encuentra ningún paquete con ese nombre en el repositorio CRAN.

La confusión es comprensible porque los dos lenguajes se usan en paralelo en el curso y las funciones tienen nombres parecidos: `read_csv` existe en ambos (en `readr` y en `pandas`), `filter` existe en `dplyr` y `query` cumple un rol similar en `pandas`. Pero los paquetes no se comparten. Cada lenguaje tiene su propio ecosistema, su propio instalador (`pip` en Python, `install.packages()` en R) y su propio repositorio (PyPI y CRAN).

28. a `V · b` `V · c` `V · d` `F`.

Cada paquete de Python del curso cumple una función específica.

`pandas` gestiona tablas de datos. Su estructura central es el `DataFrame`, y con él se leen archivos (`pd.read_csv`), se seleccionan columnas y filas, se filtran, se crean columnas nuevas, se agrupan y se resumen. Casi toda la manipulación de datos del curso se hace con `pandas`. La opción `a` es verdadera.

`numpy` gestiona arreglos numéricos. Es la base sobre la que está construido `pandas`, y aporta las operaciones vectorizadas y las funciones matemáticas. En el curso aparece de forma directa en pocas ocasiones (`np.random.seed`, `np.random.normal`, `np.clip`), pero está presente indirectamente en toda operación sobre columnas. La opción `b` es verdadera.

`matplotlib` produce gráficos. Con `plt.subplots()`, `ax.plot()`, `ax.scatter()` y `ax.set()` se construyen todos los gráficos de Python del curso. La opción `c` es verdadera.

La opción `d` es falsa. `pathlib` no hace gráficos: construye y manipula rutas de archivos. `Path.cwd()` devuelve el directorio actual, el operador `/` une componentes de una ruta y `.exists()` verifica que un archivo esté presente. Se usa en la clase 4 para leer archivos sin escribir rutas absolutas. A diferencia de los otros tres, `pathlib` forma parte de la

biblioteca estándar de Python: no requiere instalación, solo la línea `from pathlib import Path`.

29. a $V \cdot b$ V · c F · d V.

Cada paquete de R del curso cumple también una función específica.

`readr` lee archivos de texto delimitado. Su función `read_csv()` reemplaza a la `read.csv()` del R base: es más rápida, detecta automáticamente los tipos de cada columna, incluidas las fechas, y acepta un argumento `locale` para indicar la codificación. La opción `a` es verdadera.

`dplyr` transforma tablas. Sus verbos principales son `filter()` para seleccionar filas, `select()` para seleccionar columnas, `mutate()` para crear columnas, `group_by()` y `summarise()` para agrupar y resumir, y `arrange()` para ordenar. Se encadenan con el operador `%>%` o `|>`. La opción `b` es verdadera.

`ggplot2` produce gráficos, con la gramática de capas que se estudia en la clase 8. No lee archivos de ningún tipo. Los archivos Excel se leen con el paquete `readxl`, mediante `read_excel()`. La opción `c` es falsa.

`lubridate` gestiona fechas: `ymd()` convierte texto a fecha, `year()`, `month()` y `hour()` extraen componentes, `floor_date()` redondea hacia abajo a la hora o al día. `here` construye rutas relativas a la raíz del proyecto, de modo que el script funcione en cualquier máquina que tenga clonado el repositorio. La opción `d` es verdadera.

30. a F · b V · c V · d F.

Los mensajes `ModuleNotFoundError: No module named 'pandas'` en Python y `Error in library(ggplot2) : there is no package called 'ggplot2'` en R tienen la misma causa: el paquete no está instalado en el entorno que se está usando. La opción `b` es verdadera y la `a` es falsa, porque no se trata de un error de sintaxis; la línea `import pandas as pd` está bien escrita, pero el intérprete no encuentra el paquete al que se refiere.

La solución es instalar el paquete: `pip install pandas` en la terminal, o `install.packages("ggplot2")` en la consola de R. Una vez instalado, la misma línea de carga funciona sin modificaciones. La opción `c` es verdadera.

La opción `d` es falsa. Reescribir la línea de carga no cambia nada, porque el problema no está en la línea sino en la ausencia del paquete. La única excepción es que el nombre esté mal escrito, como en la pregunta 26, en cuyo caso el mensaje señala el nombre incorrecto y corregirlo resuelve el problema.

Hay un caso particular que merece atención: el paquete está instalado, pero en otro entorno. Python permite tener varios entornos virtuales en la misma máquina, cada uno con sus propios paquetes. Si `pandas` se instaló en un entorno y el script se ejecuta con otro, aparece el mismo mensaje. El indicio es que la instalación parece haber funcionado y el error persiste. En ese caso hay que verificar qué intérprete está usando el editor (VS Code lo muestra en la barra inferior) y activar el entorno correcto, o instalar el paquete en el entorno que efectivamente se usa.

31. a V · b V · c V · d F.

La línea `import pandas as pd` tiene dos partes. `import pandas` carga el paquete y lo hace

disponible bajo el nombre `pandas`. La cláusula `as pd` define un alias: a partir de esa línea, el paquete puede referirse como `pd`. El único propósito del alias es abreviar la escritura, dado que el nombre del paquete aparece muchas veces en cualquier script. La opción a es verdadera.

Con el alias, las funciones se invocan anteponiendo `pd.`: `pd.read_csv()`, `pd.DataFrame()`, `pd.to_datetime()`. La opción b es verdadera.

La opción c es verdadera. La importación sin alias, `import pandas`, es igualmente válida. La diferencia es que las funciones deben invocarse con el nombre completo: `pandas.read_csv()`, `pandas.DataFrame()`. El código funciona igual; solo es más largo de escribir.

La opción d es falsa. El alias `pd` no es un requisito del lenguaje; es una convención adoptada por la comunidad de usuarios de `pandas`, del mismo modo que `np` para NumPy y `plt` para `matplotlib.pyplot`. Podría usarse cualquier otro nombre, pero conviene respetar la convención: toda la documentación, los ejemplos, los foros y las respuestas de los asistentes de inteligencia artificial usan `pd`, y un script que use otro alias es más difícil de leer para cualquier otra persona. Si en un script aparece `pd.read_csv` y falla con `NameError: name 'pd' is not defined`, la causa es que la línea de importación no se ejecutó, no que el alias sea incorrecto.

Clase 4 — Rutas, carga y perfilado

32. a V · b V · c F · d V.

Una ruta es la indicación de dónde se encuentra un archivo en el disco. Para el lenguaje de programación, una ruta es simplemente una cadena de texto — `"data/raw/datos.csv"` — que ciertas funciones saben interpretar como ubicación. La opción d es verdadera.

Hay dos tipos de ruta. La ruta absoluta indica la ubicación completa desde la raíz del disco: `"C:/Users/jodavyt/Documents/GitHub/proyecto/data/raw/datos.csv"` en Windows, `"/home/ana/proyecto/data/raw/datos.csv"` en Linux. Contiene el nombre de usuario y la estructura de carpetas de la máquina en que se escribió. Por ese motivo, un script que usa rutas absolutas funciona únicamente en esa máquina: en cualquier otra, la carpeta `C:/Users/jodavyt` no existe y la lectura falla con `FileNotFoundError`. La opción b es verdadera.

La ruta relativa indica la ubicación a partir del directorio de trabajo: `"data/raw/datos.csv"` significa “dentro del directorio actual, la carpeta `data`, dentro de ella `raw`, y allí el archivo”. Funciona en cualquier máquina que tenga la misma estructura de carpetas, que es exactamente lo que garantiza clonar un repositorio. La opción a es verdadera.

La opción c es falsa e invierte la relación. La ruta absoluta es la menos reproducible de las dos, porque ata el script a una máquina concreta. El curso exige rutas relativas precisamente para que el análisis pueda ejecutarse en la máquina de cualquier integrante del equipo, en la del docente y en cualquier otra, sin modificaciones.

33. a V · b V · c F · d V.

Todo programa se ejecuta desde algún lugar del disco: el directorio de trabajo. Las

rutas relativas se interpretan a partir de él, de modo que conocerlo es indispensable para saber dónde va a buscar el programa los archivos.

En R, `getwd()` devuelve el directorio de trabajo actual. En Python, `os.getcwd()` o `Path.cwd()` hacen lo mismo. Las opciones a y b son verdaderas. Cuando una lectura falla con `FileNotFoundError`, ejecutar esta función es el primer paso del diagnóstico: con frecuencia el directorio de trabajo no es el que se suponía (por ejemplo, es la carpeta del usuario en lugar de la carpeta del proyecto) y la ruta relativa apunta a un lugar incorrecto.

`setwd("C:/Users/jodavyt/...")` en R, o `os.chdir(...)` en Python, cambia el directorio de trabajo. Resuelve el problema en la máquina de quien escribe el script, pero lo hace fijando una ruta absoluta, con la limitación descrita en la pregunta anterior: el script deja de funcionar en cualquier otra máquina. La opción c es falsa; `setwd()` no genera rutas relativas, sino que introduce una absoluta.

`here()`, del paquete del mismo nombre, resuelve el problema de otra manera. Busca hacia arriba en el árbol de carpetas hasta encontrar un marcador de raíz de proyecto —un archivo `.Rproj`, una carpeta `.git`— y construye las rutas a partir de allí: `here("clases", "clase4", "archivo.csv")`. El resultado es una ruta correcta en cualquier máquina que tenga el proyecto clonado, sin que el script contenga ninguna ruta absoluta. La opción d es verdadera. En Python, el script de la clase 7 define una función `raiz_proyecto()` que hace exactamente lo mismo.

34. a V · b V · c V · d F.

`pathlib` es el módulo de Python para trabajar con rutas de archivos. Forma parte de la biblioteca estándar, de modo que no requiere instalación; se carga con `from pathlib import Path`.

`Path.cwd()` devuelve el directorio de trabajo actual como un objeto `Path`. La opción a es verdadera.

El operador `/` une los componentes de una ruta. En la clase 4, `BASE_DIR / "clases" / "clase4" / archivo` construye la ruta al archivo a partir del directorio base. Esta sintaxis reemplaza la concatenación de cadenas de texto con barras, que es propensa a errores (barras dobles, barras faltantes, barras en el sentido equivocado). La opción b es verdadera.

El método `.exists()` devuelve `True` si el archivo o carpeta existe. En el script de la clase se usa antes de leer: `if not ruta_csv.exists(): raise FileNotFoundError(...)`. Verificar la existencia antes de leer permite dar un mensaje de error claro, que incluya la ruta completa que se intentó, en lugar del mensaje genérico de pandas. La opción c es verdadera.

La opción d es falsa. `pathlib` funciona de manera idéntica en Windows, macOS y Linux. Una de sus ventajas es que se encarga del separador de directorios de cada sistema: Windows usa la barra invertida y los demás la barra común, y `pathlib` produce la correcta sin que el programador tenga que ocuparse. Un script escrito con `pathlib` en Windows funciona sin cambios en Mac.

35. a V · b V · c V · d F.

El perfilado de datos es la inspección sistemática de un conjunto de datos recién cargado, antes de aplicarle cualquier modificación. Las funciones que se usan en esta etapa tienen equivalentes directos entre R y Python.

`df.info()` en pandas muestra el número de filas, el nombre y tipo de cada columna y la cantidad de valores no nulos por columna. Es el equivalente de `str(df)` en R, que muestra la estructura del data frame con el tipo y los primeros valores de cada columna. La opción a es verdadera.

`df["col"].nunique()` devuelve la cantidad de valores distintos de una columna. Es el equivalente de `length(unique(df$col))` en R. Se usa para identificar la columna identificadora, como se ve en la pregunta siguiente. La opción b es verdadera.

`df.shape` devuelve una tupla con el número de filas y el número de columnas, en ese orden: (1222, 41). En R, `dim(df)` devuelve lo mismo. La opción c es verdadera.

La opción d es falsa. `df.describe()` calcula estadísticos descriptivos de las columnas numéricas: cantidad, media, desviación estándar, mínimo, cuartiles y máximo. Es el equivalente de `summary(df)` en R. No cuenta filas duplicadas; para eso se usa `df.duplicated().sum()` en pandas o `sum(duplicated(df))` en R.

36. a V · b F · c V · d V.

La columna identificadora es aquella que identifica de forma única a cada fila de la tabla. Su propiedad definitoria es que tiene tantos valores distintos como filas: si la tabla tiene 1222 filas y la columna `matricula_letra` tiene 1222 valores distintos, cada fila tiene un valor propio y esa columna es un identificador. Si tiene menos, hay filas que comparten valor y la columna no identifica. Por eso el procedimiento de la clase 4 consiste en comparar `nunique()` con el número de filas. La opción a es verdadera.

La opción b es falsa. El identificador puede estar en cualquier posición de la tabla. En muchos archivos aparece en la primera columna por convención, pero en otros está en el medio o al final, y en algunos no hay ninguna columna que por sí sola identifique las filas.

La opción c es verdadera y es el motivo por el que el identificador importa. Cuando se quiere combinar información de dos tablas —por ejemplo, las mediciones de ozono con las de material particulado—, la unión se hace apareando las filas que tienen el mismo valor en la columna identificadora. Sin identificador no hay forma de saber qué fila de una tabla corresponde a qué fila de la otra.

La opción d es verdadera. En el ejemplo del curso, ninguna columna identifica por sí sola cada medición: la misma fecha aparece en varias estaciones y la misma estación aparece en muchas fechas. La observación queda identificada por la combinación de fecha y estación, que es un identificador compuesto. En la unión de la clase 9, la clave es exactamente esa combinación.

37. a V · b F · c V · d V.

El perfilado incluye la detección de valores faltantes y de filas duplicadas.

`df.isnull().sum()` (o `df.isna().sum()`, que es equivalente) devuelve la cantidad de valores faltantes en cada columna. En R, `colSums(is.na(df))` hace lo mismo. La opción a es verdadera.

La opción b es falsa y señala un problema frecuente en archivos de datos abiertos. pandas reconoce como faltante el valor NaN y, al leer un CSV, convierte a NaN las celdas vacías y algunas cadenas convencionales como "NA" o "null". Pero una celda que contiene un espacio, un guion, la cadena "N/A" con barra, "s/d" o "sin dato" es, para pandas, un texto válido, y no se cuenta como faltante. Son los llamados falsos nulos. El conteo de `isnull()` los pasa por alto, y una columna que parece completa puede tener cientos de ellos. Hay que buscarlos por separado, por ejemplo con `(df == "").sum()` o `(df == "N/A").sum()`, y luego convertirlos a NaN explícitamente. En R, el script de la clase 4 los busca con `sapply(df, function(x) sum(x == "" | x == "N/A", na.rm = TRUE))`.

`df.duplicated().sum()` devuelve la cantidad de filas que son exactamente iguales a otra fila anterior. `df.drop_duplicates()` devuelve la tabla sin esas repeticiones, conservando la primera aparición de cada fila. Las opciones c y d son verdaderas. En R, `sum(duplicated(df))` y `distinct(df)` cumplen las mismas funciones. Antes de eliminar duplicados conviene examinarlos, porque a veces la repetición es legítima (dos mediciones idénticas en instantes distintos) y a veces indica un problema en la fuente.

38. a V · b V · c F · d V.

La unidad de observación es aquello que representa cada fila de la tabla: el objeto de estudio al que corresponde cada registro. Definirla es el primer paso del perfilado, porque determina qué significan las columnas, cuál es el identificador y qué análisis tienen sentido. La opción a es verdadera.

Un mismo dominio admite varias unidades de observación, y el ejemplo de los videojuegos de la clase 4 lo ilustra. En una tabla a nivel de acción, cada fila es un evento dentro del juego, identificado por `event_id`. En una tabla a nivel de partida, cada fila es una partida o un jugador en una partida, con `match_id` y `player_id`, y columnas como `kills`, `deaths` o `win_bool`. En una tabla a nivel de jugador, cada fila es un jugador único, con su gasto acumulado y su retención a 7 o 30 días. En una tabla a nivel de título, cada fila es un videojuego, con `app_id`, `genre`, `price` y usuarios concurrentes. Son cuatro tablas distintas, con identificadores distintos, para preguntas distintas. Las opciones b y d son verdaderas.

La opción c es falsa. La unidad de observación es determinante para unir tablas, porque define cuál es la columna por la que se aparean las filas. Unir una tabla a nivel de partida con una tabla a nivel de jugador exige agregar primero la primera al nivel de jugador, o repetir la información del jugador en cada partida; en cualquier caso, hay que decidir en qué nivel se trabaja. Una unión entre tablas de distinto nivel sin esa decisión previa produce filas duplicadas o información perdida sin que ningún mensaje de error lo advierta.

Clase 5 — Flujo de análisis y formatos

39. a V · b F · c V · d V.

El flujo de análisis de datos presentado en la clase 5 tiene seis etapas.

La ingesta o carga es la captura de los datos crudos desde sus fuentes hacia el entorno

de trabajo. Su objetivo es garantizar la disponibilidad de los datos registrando su origen exacto, sin modificarlos. La opción d es verdadera: en esta etapa los datos originales no se alteran, precisamente para que siempre sea posible volver a ellos.

El perfilado es la inspección sistemática de los datos recién cargados: conteo de nulos, cardinalidad (cantidad de valores únicos por columna), distribuciones de frecuencia, detección de tipos inconsistentes y de valores fuera de rango, identificación de formatos erróneos (fechas mezcladas, teléfonos mal estructurados). La opción c es verdadera.

La limpieza y preparación aplica las correcciones que el perfilado identificó: conversión de tipos, tratamiento de faltantes, deduplicación, normalización de nombres. La exploración calcula estadísticos descriptivos y examina relaciones entre variables. La transformación crea variables derivadas y agrega a distintos niveles. La entrega distribuye los resultados a sus destinos: tablas, cuadros de mando, informes. La opción a resume el orden y es verdadera.

La opción b es falsa e invierte el orden de dos etapas. El perfilado se realiza antes de la limpieza, no después. Su propósito es diagnosticar: saber qué hay en los datos antes de decidir qué hacer con ellos. Limpiar sin perfilar significa aplicar reglas a ciegas, y con frecuencia elimina información válida o deja pasar problemas que no se buscaron. El perfilado debe culminar con un código definitivo de carga, que documente el tipo de conexión, el formato, los tipos de dato y la completitud.

40. a F · b V · c V · d V.

Los datos llegan en distintos formatos, y cada uno tiene propiedades que condicionan cómo se leen y qué se puede confiar de ellos.

CSV, TSV y TXT son formatos de texto plano con un separador definido: coma, tabulador o punto y coma. Son livianos, universales y legibles por cualquier sistema. Pero no guardan tipos de dato: un CSV no sabe si la columna fecha es una fecha o un texto, ni si precio es un número. El programa que lo lee tiene que inferir o recibir esa información. Tampoco guardan reglas de integridad ni metadatos. La opción a es falsa.

Excel (.xlsx) admite múltiples hojas y formato visual, y está muy extendido en la gestión administrativa. Tiene un máximo fijo de filas por hoja, 1 048 576, y bajo rendimiento con volúmenes grandes. La opción d es verdadera.

JSON organiza los datos en pares clave-valor y arreglos anidados. Es flexible, admite estructuras jerárquicas y es el formato nativo de la web y de las APIs modernas. La opción c es verdadera.

Parquet es un formato binario, columnar y comprimido. Almacena los datos por columna en lugar de por fila, lo que permite leer solo las columnas necesarias para una consulta, y guarda los tipos de dato. Está diseñado para analítica sobre volúmenes grandes. La opción b es verdadera. Junto con Avro (orientado a filas, con esquema en JSON) y ORC (columnar comprimido), forma la familia de formatos masivos.

41. a F · b V · c F · d V.

La clase 5 planteó un debate: ¿es Excel una base de datos? La respuesta es que no, y la comparación con PostgreSQL muestra por qué.

Excel es una hoja de cálculo. Está diseñado para la edición manual, el cálculo reactivo celda a celda y la presentación visual de datos de pequeña escala. No fuerza relaciones entre tablas: no hay forma de exigir que cada `cliente_id` de la hoja de ventas exista en la hoja de clientes. Una misma columna puede contener números, textos y fechas mezclados sin que nada lo impida. No soporta acceso simultáneo de escritura de muchos usuarios y tiene un máximo fijo de filas. Las opciones a y c son falsas.

PostgreSQL es un sistema de gestión de bases de datos relacionales. Exige esquemas: cada columna tiene un tipo de dato único y declarado, y un valor de otro tipo es rechazado. Admite claves primarias, que garantizan la unicidad del identificador, y claves foráneas, que garantizan la integridad referencial entre tablas. Gestiona transacciones con propiedades ACID (atomicidad, consistencia, aislamiento, durabilidad) y maneja miles de conexiones simultáneas de lectura y escritura sin corromper los datos. Las opciones b y d son verdaderas.

La distinción importa para el curso porque los equipos que trabajan con datos transaccionales van a leer el modelo de datos de un sistema de gestión y a consultarlo con SQL. Entender que una base de datos impone restricciones que Excel no impone es lo que permite confiar en la integridad de lo que se lee.

42. a V · b F · c V · d F.

El bloque de la clase 5 resuelve un problema recurrente con los archivos del Catálogo de Datos Abiertos: no todos están codificados en UTF-8, y leerlos con la codificación incorrecta produce un error o caracteres corruptos.

La estructura es un `try / except`. Dentro del `try` se intenta la lectura con `encoding="utf-8"`. Si el archivo está en esa codificación, la lectura funciona y el bloque termina. Si no lo está, pandas produce `UnicodeDecodeError`; el bloque `except UnicodeDecodeError` captura ese error específico, imprime un aviso y reintenta la lectura con `encoding="latin-1"`. La opción a es verdadera.

La opción b es falsa. `UnicodeDecodeError` no es un error de tipo (`TypeError`). Es un error de decodificación: los bytes del archivo no pueden interpretarse con la tabla de caracteres indicada. Cada codificación asigna secuencias de bytes a caracteres, y una secuencia válida en latin-1 puede no serlo en UTF-8.

La opción d es falsa. El bloque `except` no borra el archivo ni lo modifica. Solo ejecuta una lectura alternativa. El archivo permanece intacto en el disco.

La opción c se refiere al pseudocódigo con que la clase presentó este bloque antes de escribirlo en Python. El pseudocódigo describe la lógica en lenguaje natural estructurado (INTENTAR, SI FALLA ENTONCES, FIN INTENTAR) sin comprometerse con la sintaxis de ningún lenguaje. Permite diseñar el control de errores antes de enfrentarse a las comas, los dos puntos y la indentación, y sirve como mapa para la implementación. La opción c es verdadera.

43. a V · b V · c F · d V.

La clase 5 distinguió dos estrategias para cargar datos desde una fuente que se

actualiza.

La carga inicial o completa (*full load*) lee e importa el volumen total de datos disponible. Se usa para poblar una base nueva, para migraciones completas o para reiniciar el almacenamiento desde cero. Su ventaja es la consistencia: no depende de ningún estado previo. Su desventaja es el costo en tiempo, ancho de banda y cómputo cuando el volumen es grande. La opción a es verdadera.

La carga incremental identifica e importa únicamente los registros creados, modificados o eliminados desde la última ejecución. Para detectarlos existen varias estrategias: marcas de tiempo de actualización en cada registro, secuencias numéricas crecientes, o captura de cambios en la base de datos (CDC, *change data capture*), que registra cada modificación a medida que ocurre. Las opciones b y d son verdaderas.

La opción c es falsa. La carga incremental es más rápida y consume menos recursos, pero es más compleja de implementar, no más simple. Hay que mantener un registro de hasta dónde se cargó la última vez, decidir qué hacer con los registros modificados (reemplazar, versionar), evitar que un registro se cargue dos veces y detectar los eliminados. Un error en esa lógica produce duplicaciones o inconsistencias que son difíciles de detectar. La carga completa, por comparación, no tiene estado que mantener.

44. a V · b V · c F · d V.

El ejercicio de pseudocódigo de la clase 5 pedía filtrar las temperaturas mayores a 20 del vector 18, 22, 15, 25, 30. El pseudocódigo lo resolvía con un bucle: recorrer cada elemento, comparar, y agregar al resultado los que cumplen. La traducción a código muestra que ninguno de los dos lenguajes necesita el bucle explícito.

En R, la expresión `temperaturas > 20` no devuelve los valores mayores a 20. Devuelve un vector lógico del mismo largo que el original, con un TRUE o FALSE por elemento: FALSE TRUE FALSE TRUE. Es la misma máscara booleana de la pregunta 20. La opción c es falsa.

Para obtener los valores, se usa esa máscara como índice: `temperaturas[temperaturas > 20]` devuelve los elementos en las posiciones donde la máscara es TRUE, es decir, 22 25 30. Es el filtrado vectorizado nativo de R. Las opciones a y d son verdaderas.

En Python, con listas, la herramienta equivalente es la comprensión de lista: `[t for t in temperaturas if t > 20]`. Se lee “cada t de temperaturas tal que t sea mayor a 20”, y devuelve [22, 25, 30]. Es un bucle compacto, escrito en una línea. La opción b es verdadera. Con un arreglo de NumPy o una columna de pandas se usaría la sintaxis de máscara, igual que en R: `temperaturas[temperaturas > 20]`.

Clase 6 — Medición y diseño de encuestas

45. a V · b F · c V · d V.

Stevens (1946) propuso clasificar las variables según el tipo de operaciones que admiten sus valores. La clasificación tiene cuatro niveles, cada uno con las propiedades del anterior más una adicional. La opción a es verdadera.

El nivel nominal distingue categorías sin orden: textura, color, departamento de residencia. Solo se puede decir si dos valores son iguales o distintos.

El nivel ordinal agrega el orden: rigidez, nivel educativo, satisfacción de uno a cinco. Se puede decir que un valor es mayor que otro, pero no cuánto mayor: la distancia entre “satisfecho” y “muy satisfecho” no está definida ni es necesariamente igual a la distancia entre “insatisfecho” y “neutral”. La opción d es verdadera.

El nivel de intervalo agrega distancias iguales entre valores, pero el cero es convencional. La temperatura en grados Celsius es el ejemplo clásico: la diferencia entre 10 °C y 20 °C es la misma que entre 20 °C y 30 °C, pero 0 °C no es ausencia de temperatura, y por lo tanto 20 °C no es “el doble de caliente” que 10 °C. Los cocientes no tienen sentido. La opción b es falsa.

El nivel de razón agrega un cero absoluto que representa ausencia de la magnitud. El ingreso en pesos, el peso en gramos, la cantidad de hijos: cero significa nada, y los cocientes tienen sentido (un ingreso de 40 000 es el doble de uno de 20 000). La opción c es verdadera. La clasificación importa porque determina qué estadísticos son válidos: la media tiene sentido para intervalo y razón, la mediana desde ordinal, la moda para todos.

46. a V · b V · c F · d V.

La clase 6 distinguió tres formas en que se generan los datos sobre la conducta de las personas.

La preferencia declarada es lo que la persona dice: la respuesta a una encuesta, una entrevista, un formulario. Se obtiene preguntando. La opción a es verdadera.

La preferencia revelada es lo que la persona hace, inferido de sus acciones observables: una compra registrada en un sistema transaccional, un producto elegido entre varios, un servicio contratado. Se obtiene observando. La opción b es verdadera.

La huella digital es el registro automático de la conducta que dejan los sistemas informáticos: clics, tiempo en una página, ubicación del teléfono, historial de búsqueda. No requiere que la persona responda ni que un observador registre; el sistema lo hace por diseño. La opción d es verdadera.

La opción c es falsa. La preferencia declarada no es intrínsecamente más confiable que la revelada; con frecuencia es menos confiable, porque lo que una persona declara y lo que efectivamente hace pueden diferir por muchos motivos: deseabilidad social, memoria imperfecta, falta de reflexión sobre la propia conducta, intención de dar una imagen determinada. Una encuesta sobre consumo de alcohol y el registro de ventas de una licorería cuentan historias distintas. Ninguna de las tres fuentes es superior en general: cada una mide algo distinto, con sus propios sesgos, y la elección depende de la pregunta.

47. a V · b V · c F · d V.

La Ley 18.331 de Protección de Datos Personales de Uruguay regula el tratamiento de la información sobre personas identificadas o identificables. Dentro de los datos personales, la ley distingue una categoría con protección reforzada: los

datos sensibles. Son los que revelan origen racial o étnico, preferencias políticas, convicciones religiosas o morales, afiliación sindical, e información referente a la salud o a la vida sexual. Su tratamiento requiere consentimiento expreso y escrito, y en ciertos casos está prohibido. Las opciones a y b son verdaderas.

La opción c es falsa. El precio de un producto no es un dato personal: no se refiere a una persona. Los datos sobre objetos, empresas, precios o lugares no están alcanzados por la ley, salvo que permitan identificar a una persona.

La opción d es verdadera y es el punto que más importa para el trabajo con datos. Suprimir el nombre y el documento de identidad no convierte un conjunto de datos en anónimo. La combinación de unas pocas variables aparentemente inocuas —fecha de nacimiento, código postal, sexo, ocupación— identifica de forma única a una gran proporción de la población. Es el riesgo de reidentificación: cruzando el conjunto “anonimizado” con otra fuente pública, se recupera la identidad. Por ese motivo, los equipos que trabajan con microdatos de encuestas deben tratar los datos como personales aunque no contengan nombres, y no versionar en el repositorio ningún archivo que permita identificar a una persona.

48. a V · b V · c F · d V.

El diseño de un instrumento de medición —un cuestionario, una escala, un sensor— se evalúa con dos criterios distintos.

La validez es la propiedad de medir efectivamente lo que se pretende medir. Un cuestionario que busca medir satisfacción con un servicio es válido si sus respuestas reflejan la satisfacción y no otra cosa: la simpatía del encuestador, el deseo de terminar rápido, la interpretación ambigua de una pregunta. La opción a es verdadera.

La confiabilidad es la propiedad de producir resultados consistentes al repetir la medición en condiciones equivalentes. Un instrumento confiable da el mismo resultado si se aplica dos veces a la misma persona en el mismo estado. La opción b es verdadera.

La opción c es falsa. Validez y confiabilidad son independientes, y el ejemplo de la balanza lo muestra. Una balanza descalibrada que marca siempre dos kilogramos de más es perfectamente confiable —cada vez que se pesa el mismo objeto, marca lo mismo— y no es válida, porque no mide el peso real. A la inversa, una balanza que marca el peso correcto en promedio pero con gran variación entre pesadas es válida en promedio pero no confiable. Un buen instrumento necesita ambas propiedades.

La opción d define la operacionalización, que es el paso previo al diseño del instrumento. Un concepto abstracto —satisfacción, pobreza, calidad del aire— no se mide directamente; hay que traducirlo en variables observables y medibles. La calidad del aire se operacionaliza como concentración de ozono y de material particulado en microgramos por metro cúbico, medida por estaciones en ubicaciones determinadas. La operacionalización es una decisión, y decisiones distintas producen datos distintos sobre el mismo concepto. La opción d es verdadera.

49. a V · b V · c F · d V.

En una encuesta, los valores faltantes tienen orígenes distintos, y distinguirlos es indispensable para tratarlos correctamente.

El faltante estructural se produce cuando la pregunta no era aplicable al encuestado. Si el cuestionario pregunta “¿tiene hijos?” y luego “¿cuántos?”, quien respondió que no a la primera no responde la segunda. Ese vacío no es información perdida: es la consecuencia lógica de la respuesta anterior. La opción a es verdadera.

La no respuesta se produce cuando la pregunta era aplicable y no fue contestada: la persona se negó, no supo, saltó la pregunta por error, o el encuestador no la registró. Ese vacío sí es información perdida, y su tratamiento depende de por qué se produjo: si quienes no respondieron difieren sistemáticamente de quienes respondieron, ignorar los faltantes sesga el resultado. La opción b es verdadera.

La opción c es falsa. Los dos tipos no se tratan igual. Imputar un valor a un faltante estructural es un error —no tiene sentido estimar cuántos hijos tiene alguien que no tiene hijos—, y eliminar las filas con faltantes estructurales elimina a todas las personas sin hijos del análisis. El faltante estructural se codifica con un valor distinto del de la no respuesta, o se maneja mediante el diseño del análisis.

La opción d es verdadera. El libro de códigos es el documento que acompaña a un conjunto de datos y describe cada variable: su nombre, la pregunta de la que proviene, su tipo, los valores que puede tomar y qué significa cada uno, y los códigos usados para los distintos tipos de faltante (por ejemplo, 98 para “no sabe”, 99 para “no contesta”, y un código distinto para “no aplica”). Sin el libro de códigos, un archivo de microdatos es una tabla de números sin significado.

Clase 7 — Carga, limpieza, fechas y agregación

50. a $V \cdot b$ $V \cdot c$ $F \cdot d$ V .

Un archivo de texto es una secuencia de bytes, y la codificación es la tabla que indica qué carácter corresponde a cada secuencia. Las dos codificaciones que aparecen en el curso son UTF-8, el estándar actual, y latin-1 (ISO-8859-1), frecuente en archivos generados por sistemas antiguos o por Windows en español. Para los caracteres del alfabeto inglés ambas coinciden; para las vocales acentuadas, la ñe y otros caracteres, difieren.

El argumento `encoding="latin1"` en `pandas`, o `locale = locale(encoding = "latin1")` en `readr`, es una instrucción de lectura: le indica al programa qué tabla usar para interpretar los bytes. No es una propiedad que el programa pueda deducir con certeza del archivo, porque el archivo no declara su codificación. La opción a es verdadera.

Cuando la codificación de lectura no coincide con la de escritura, los caracteres fuera del rango inglés se corrompen de forma característica. Un archivo escrito en UTF-8 y leído como latin-1 muestra “Ã³” donde debería decir “ó”, “Ã±” donde debería decir “ñ”, “Ã¡” por “á”. Cada carácter acentuado, que en UTF-8 ocupa dos bytes, se interpreta como dos caracteres de latin-1. Ver esa secuencia es la señal inequívoca de un desajuste de codificación. La opción b es verdadera.

La opción c es falsa. Cambiar el argumento de codificación no modifica el archivo; modifica la interpretación. El archivo puede volver a leerse con otra codificación

cuantas veces sea necesario.

La opción d es verdadera. En el script de la clase 7, los nombres de estación llegaron corrompidos de este modo, y como el archivo original no podía corregirse, se aplicó un diccionario de reemplazos —"Ã³" por "o", "Ã±" por "ni", y así sucesivamente— mediante `str.replace()` en pandas y `str_replace_all()` en R. El orden de los reemplazos importa: la regla para "Ã" sola debe ir al final, porque de lo contrario interfiere con las reglas que contienen "Ã" seguida de otro carácter.

51. a V · b V · c F · d V.

Las fechas son el tipo de dato que más problemas produce en la carga, porque en el archivo son texto y hay que convertirlas a un tipo que el lenguaje reconozca como fecha.

`read_csv()` de `readr` inspecciona los primeros valores de cada columna e infiere el tipo; si los valores tienen forma de fecha u hora, la columna se convierte automáticamente a fecha. La opción a es verdadera. `pd.read_csv()` no lo hace por defecto: deja la columna como texto. Hay dos formas de obtener fechas en pandas: indicar `parse_dates=["fecha"]` al leer, o convertir después con `ozono["fecha"] = pd.to_datetime(ozono["fecha"])`. La opción b es verdadera.

La opción c es falsa. La clase `POSIXct` de R (cuyo nombre viene de *calendar time*) no guarda la fecha como texto sino como un número: la cantidad de segundos transcurridos desde el 1 de enero de 1970. `datetime64[ns]` de pandas hace lo mismo con nanosegundos. Esa representación numérica es lo que hace rápidas las operaciones: restar dos fechas para obtener una duración, sumar días, comparar, ordenar. Una fecha guardada como texto no admite ninguna de esas operaciones —“2024-03-07” menos “2024-03-01” no tiene sentido como resta de cadenas— y ordena mal cuando el formato no es año-mes-día. `POSIXt` es la clase virtual que agrupa a los formatos de tiempo y permite que las funciones de fecha los reconozcan.

La opción d es verdadera. Una vez que la columna es de tipo fecha, en R se extraen sus componentes con `lubridate`: `year(fecha)`, `month(fecha)`, `hour(fecha)`. En pandas, el accesor `.dt` da acceso a los mismos componentes: `fecha.dt.year`, `fecha.dt.month`, `fecha.dt.hour`. En el script, así se crean las columnas `anio`, `mes` y `hora` que luego se usan para agregar.

52. a V · b V · c V · d F.

Una vez detectados los faltantes, hay que decidir qué hacer con ellos. La decisión más simple es eliminar las filas que los contienen, y las funciones que lo hacen difieren en qué columnas examinan.

`dropna()` de pandas, sin argumentos, elimina toda fila que tenga un valor faltante en cualquier columna. Si el data frame tiene diez columnas y una fila tiene faltante en una sola, la fila completa se elimina. La opción a es verdadera. `na.omit()` de R se comporta igual: examina todas las columnas. La opción d es falsa.

Ese comportamiento es más agresivo de lo que suele desearse. En el ejemplo del curso, la única columna con faltantes es la medición `o3`; las columnas de fecha y estación están completas. Eliminar las filas sin medición es razonable, porque no aportan nada

al análisis de concentraciones. Pero si hubiera faltantes en otra columna irrelevante, `dropna()` eliminaría también esas filas, perdiendo mediciones válidas.

Para restringir la eliminación a una columna, pandas ofrece el argumento `subset`: `dropna(subset=["o3"])` elimina solo las filas cuyo valor de `o3` es faltante, sin mirar las demás columnas. La opción b es verdadera. En R, el equivalente con `dplyr` es `filter(!is.na(o3))`, que conserva las filas donde `o3` no es faltante; el script de la clase 7 lo escribe como `filter(is.na(o3) != TRUE)`, que es equivalente. La opción c es verdadera.

Antes de eliminar conviene preguntarse, como hace la sección 10 del script, cuándo y dónde faltan los valores, porque el patrón de faltantes es información sobre el proceso de medición.

53. a V · b V · c V · d F.

La agregación es la operación de resumir muchas filas en una, según un criterio de agrupamiento. Es el núcleo de la clase 7, y las herramientas de R y Python son directamente equivalentes.

En R con `dplyr`, la agregación se escribe en dos pasos: `group_by(mes)` define los grupos, y `summarise(o3_medio_mes = mean(o3), n_registros_mes = n())` calcula un estadístico por grupo. En pandas, `groupby("mes")` define los grupos y `.agg(o3_medio_mes=("o3", "mean"), n_registros_mes=("o3", "size"))` calcula los estadísticos. La opción a es verdadera.

`n()` en R y `"size"` en pandas cuentan cuántas filas hay en cada grupo. Ese conteo es tan importante como el estadístico: dice cuántas observaciones respaldan cada valor agregado. La opción b es verdadera.

Cuando el agrupamiento es temporal —por hora, por día— hay que redondear primero cada fecha al inicio de su período. En R, `floor_date(fecha, unit = "hour")` crea la columna con la hora redondeada, y luego `group_by(estacion, fecha_hora)` agrupa. En pandas, `pd.Grouper(key="fecha", freq="h")` hace las dos cosas a la vez dentro del `groupby`. La opción c es verdadera.

La opción d es falsa y es un punto donde R y Python difieren. La función `mean()` de R devuelve NA si el vector contiene algún valor faltante; para ignorarlos hay que escribir `mean(x, na.rm = TRUE)`. Lo mismo ocurre con `sum()`, `max()` y `quantile()`. El método `.mean()` de pandas, en cambio, omite los faltantes por defecto. Un `summarise()` en R que devuelve NA en todos los grupos casi siempre se debe a esto.

54. a V · b V · c V · d V.

Las cuatro afirmaciones son verdaderas. La pregunta apunta a la decisión central de la sección 5 del script: al agregar de datos minutales a horarios o diarios, ¿qué estadístico representa el fenómeno? La respuesta es que ninguno lo representa por completo, y que cada uno cuenta una historia distinta.

La media es el nivel típico del período: el valor alrededor del cual estuvieron las mediciones durante esa hora. Es el estadístico habitual y el más fácil de interpretar, pero no informa sobre los episodios de concentración alta, que son los que importan para la salud.

El máximo captura el episodio más intenso del período. Pero un sensor produce lecturas erróneas ocasionales —un pico de un minuto que no corresponde a la atmósfera—, y el máximo las toma como si fueran reales. Es el estadístico más sensible a errores de medición.

El percentil 95 es un compromiso: captura episodios de concentración alta, porque solo el 5 % de los minutos lo superan, y es robusto a picos aislados, porque un minuto erróneo entre sesenta no lo altera. El panel E del script de estadística básica muestra exactamente eso: con diez picos espurios agregados a un día de mediciones, el máximo cambia por completo y el p95 casi no se mueve.

$n_{\min}$ es la cantidad de minutos que respaldan cada valor agregado. Una hora completa tiene sesenta; un día, 1440. Si una hora tiene $n_{\min} = 12$, su media se calculó con doce minutos de datos y es menos confiable que la de una hora completa. Sin esta columna, todas las horas parecen igualmente sólidas. Es la medida de cobertura del dato agregado.

55. a V · b V · c V · d F.

La sección 10 del script examina los patrones de faltantes de una serie de sensor, y distingue tres tipos que exigen preguntas distintas.

El faltante explícito es una fila que existe en el archivo, con su fecha y su estación, pero cuya medición es NA. El sensor estaba registrando pero no produjo valor: mantenimiento, calibración, lectura descartada. Se detecta con `isna()` y se resume por estación con la proporción de NA. La opción a es verdadera.

El faltante implícito es una fila que no existe: el minuto no está en el archivo. Entre las 10:15 y las 10:17 no hay ninguna fila, y ningún conteo de NA lo va a mostrar, porque no hay nada que contar. Se detecta comparando la cantidad de minutos presentes con la cantidad esperada entre la primera y la última fecha, o contra la grilla completa del período pedido. La opción b es verdadera.

Los faltantes en rachas son secuencias largas de NA consecutivos. Un NA aislado es una lectura perdida; cien NA seguidos indican que el sensor estuvo caído durante más de una hora, y varios días seguidos indican una interrupción prolongada. El script los identifica con una función equivalente a `rle()` de R y los clasifica por duración. La opción c es verdadera.

La opción d es falsa. Eliminar los NA sin examinar cuándo y cómo faltan destruye información sobre el proceso de medición. Después de `dropna()`, ya no se puede saber que una estación estuvo fuera de servicio durante una semana de marzo, y las medias de esa semana se calculan con los pocos minutos que quedaron, sin ninguna advertencia. El análisis de faltantes es una etapa del perfilado, previa a cualquier eliminación.

56. a V · b V · c V · d F.

El script de estadística básica de la clase 7 muestra que la media y el percentil pueden leerse directamente de una tabla de frecuencias, y el histograma es la representación gráfica de esa tabla.

La tabla divide el rango de la variable en intervalos y cuenta cuántas observaciones

caen en cada uno (frecuencia absoluta, n_i), qué proporción del total representan (frecuencia relativa, $f_i = n_i / n$) y la proporción acumulada hasta ese intervalo (F_i). Cuando los intervalos tienen la misma amplitud, la altura de cada barra del histograma puede ser la frecuencia. Cuando tienen amplitudes distintas, un intervalo ancho acumula más observaciones solo por ser ancho, y la comparación de alturas engaña. La solución es la densidad: $h_i = f_i / a_i$, la frecuencia relativa dividida por la amplitud del intervalo. Con la densidad, el área de cada barra —y no su altura— es proporcional a la frecuencia, y las barras son comparables. La opción a es verdadera.

Las opciones b y c se refieren a una convención que es fácil de pasar por alto. La función `hist()` de R construye por defecto intervalos cerrados a la derecha, $(a, b]$: un valor exactamente igual a un límite se cuenta en el intervalo de la izquierda. El argumento `right = FALSE` invierte la convención a $[a, b)$, que es la que usan las etiquetas del script. La diferencia solo afecta a los valores que caen exactamente en un límite, por lo que el error no se nota, pero produce conteos distintos de los que la tabla declara. Ambas opciones son verdaderas.

La opción d es falsa. La media puede calcularse desde la tabla como promedio ponderado de las marcas de clase (los puntos medios de cada intervalo) por sus frecuencias. Pero es una aproximación: al agrupar, se pierde la posición exacta de cada dato dentro de su intervalo, y se lo reemplaza por el punto medio. La media exacta requiere los datos individuales. Lo mismo vale para el percentil 95, que desde la tabla solo puede ubicarse en un intervalo, no en un valor.

Clase 8 — Visualización

57. a $V \cdot b$ F $\cdot c$ $V \cdot d$ V.

El teórico de la clase 8 parte del gráfico de ozono mensual producido en la clase 7 y lo descompone en sus partes.

Todo gráfico tiene cinco componentes básicos. El conjunto de datos del que proviene; la variable que va en el eje horizontal; la variable que va en el eje vertical; el tipo de gráfico o geometría (línea, puntos, barras); y los títulos: el del gráfico y los rótulos de los ejes. En `ggplot2`, esos cinco componentes se corresponden con `ggplot(datos, aes(x = ..., y = ...))`, la geometría `geom_*()` y `labs()`. En `matplotlib`, con los argumentos de `ax.plot()` y de `ax.set()`. La opción a es verdadera.

La opción b es falsa. El gráfico de partida tenía el eje vertical rotulado como "Valores", y ese fue el primer defecto señalado. El rótulo no dice qué magnitud se representa ni en qué unidad. Quien mira el gráfico ve una curva entre 24 y 36 y no sabe si son microgramos, partes por millón o un índice. El rótulo adecuado es Ozono ($\mu\text{g}/\text{m}^3$): nombra la variable e incluye la unidad. La opción c es verdadera.

La opción d es verdadera y distingue dos tipos de decisión. Fijar los límites del eje vertical con `ylim(0, 40)` no modifica ningún dato: los cuatro valores mensuales siguen siendo los mismos. Lo que cambia es cómo se leen. Con el eje desde cero, las diferencias entre meses se ven en proporción a la magnitud total; con el eje desde 24 (que es lo que hace el gráfico automático), las mismas diferencias parecen enormes. Es una decisión de lectura que el analista toma y debe poder justificar.

58. a V · b V · c V · d F.

Cada tipo de gráfico responde a un tipo de pregunta, y elegir el tipo es elegir qué se quiere mostrar.

El gráfico de líneas conecta observaciones sucesivas y es adecuado cuando el eje horizontal tiene un orden natural, típicamente el tiempo. La línea sugiere continuidad entre los puntos, lo que tiene sentido para una serie temporal y no para categorías inconexas. La opción a es verdadera.

El gráfico de barras compara un valor entre categorías: la concentración media de ozono en cada estación, la cantidad de registros por mes. Cada barra es independiente de las demás y el orden puede elegirse. La opción b es verdadera.

El gráfico de dispersión muestra cada observación como un punto en el plano definido por dos variables numéricas, y sirve para examinar si hay relación entre ellas: si cuando una es alta la otra también lo es, si hay una nube sin estructura, si hay grupos. Es el gráfico de la sección 7 del script de la clase 9, ozono contra PM2.5. La opción c es verdadera.

La opción d es falsa. La distribución de una variable —cómo se reparten sus valores, dónde se concentran, qué tan dispersos están— se muestra con un histograma o un diagrama de caja, que son los gráficos de la pregunta siguiente. Un gráfico de líneas no muestra la distribución: muestra la evolución. Elegir el tipo incorrecto no produce error; produce un gráfico que responde otra pregunta.

59. a V · b V · c V · d F.

El histograma y el diagrama de caja son los dos gráficos básicos para examinar la distribución de una variable numérica.

El histograma divide el rango de la variable en intervalos y dibuja una barra por intervalo cuya altura (o área, si se usa densidad) indica cuántas observaciones caen en él. Muestra la forma de la distribución: si es simétrica o asimétrica, si tiene uno o varios modos, dónde se concentran los valores. En el script de la clase 7, el histograma de las mediciones de un día mostró una distribución asimétrica a la derecha, como corresponde a una concentración. La opción a es verdadera.

El diagrama de caja resume la distribución en cinco números: la mediana (línea central), el primer y tercer cuartil (bordes de la caja, que encierra el 50 % central de los datos), y los valores extremos (bigotes), con los puntos que quedan más allá marcados como atípicos. Es más compacto que el histograma y permite comparar varias distribuciones lado a lado: en la clase 9, un boxplot por mes muestra cómo cambió la distribución del ozono de enero a abril. La opción b es verdadera.

Ambos gráficos describen una sola variable en un momento dado o en un período; no muestran cómo esa variable evolucionó en el tiempo. Para eso está la línea. La opción c es verdadera y la d es falsa: el histograma necesita una única variable, cuyos valores se agrupan en intervalos. Cuando se dibuja un histograma por grupo (uno por estación), la segunda variable define los grupos, no el eje.

60. a V · b V · c V · d F.

ggplot2 está construido sobre una gramática de gráficos: un gráfico es la suma de componentes, cada uno especificado por separado, y el operador + los combina.

ggplot(datos, aes(x = mes, y = o3_medio_mes)) es el primer componente y establece dos cosas: el conjunto de datos que se va a representar, y el mapeo estético (aes, de *aesthetics*): qué variable va en cada eje. Por sí solo no dibuja nada; produce un lienzo vacío con los ejes definidos. La opción a es verdadera.

+ geom_line() agrega una capa geométrica: dibuja una línea que conecta los puntos definidos por el mapeo. + geom_point() agregaría otra capa con los puntos. Cada geom_*() es una capa, y pueden sumarse tantas como se quiera. La opción b es verdadera.

+ labs(title = "...", x = "...", y = "...") establece el título y los rótulos de los ejes. Sin labs(), ggplot2 usa los nombres de las columnas como rótulos, lo que rara vez es adecuado para un lector externo. La opción c es verdadera.

La opción d es falsa. geom_line() sola, sin ggplot() antes, no produce un gráfico: no tiene datos que representar ni sabe qué variable va en cada eje. La secuencia siempre empieza con ggplot(), y las geometrías se suman a ella. Escribir geom_line(datos) es un error frecuente en las primeras semanas y produce un mensaje sobre el argumento data que no ayuda a entender el problema.

61. a V · b V · c V · d V.

Las cuatro afirmaciones son verdaderas y describen la secuencia con que se construye cualquier gráfico de matplotlib en el curso.

fig, ax = plt.subplots(figsize=(8, 5)) crea dos objetos: la figura (fig), que es el lienzo completo y el que se guarda en un archivo, y los ejes (ax), que es el área donde se dibuja. La distinción importa cuando hay varios paneles: una figura contiene varios ejes. El argumento figsize fija el tamaño en pulgadas.

ax.plot(x, y) dibuja una línea que conecta los puntos (x, y) en el orden en que aparecen. Admite argumentos de estilo como color, linewidth y label.

ax.scatter(x, y) dibuja un punto por cada par (x, y), sin conectarlos. Admite color, s (tamaño) y zorder (orden de superposición).

ax.set(title="...", xlabel="...", ylabel="...") establece el título y los rótulos de los ejes en una sola llamada. Equivale a llamar por separado a ax.set_title(), ax.set_xlabel() y ax.set_ylabel().

A esa secuencia se agregan, según el caso, ax.grid(), ax.legend(), plt.tight_layout() para ajustar los márgenes, y plt.show() para mostrar o fig.savefig() para guardar. La estructura es siempre la misma: crear figura y ejes, dibujar sobre los ejes, rotular, mostrar o guardar.

62. a V · b V · c V · d F.

Como el curso usa ggplot2 y matplotlib en paralelo, conviene tener presentes las equivalencias entre geometrías.

geom_line() de ggplot2 corresponde a ax.plot() de matplotlib: ambas conectan puntos sucesivos con una línea. La opción a es verdadera.

`geom_point()` corresponde a `ax.scatter()`: ambas dibujan un punto por observación. La opción b es verdadera. (En `matplotlib`, `ax.plot(x, y, "o")` también dibuja puntos sin línea, pero `scatter` ofrece más control sobre tamaño y color por punto.)

`geom_histogram()` corresponde a `ax.hist()`: ambas agrupan los valores en intervalos y dibujan una barra por intervalo. La opción c es verdadera.

La opción d es falsa. `geom_boxplot()` corresponde a `ax.boxplot()`, no a `ax.plot()`. `ax.plot()` dibuja líneas; el diagrama de caja tiene su propia función, que calcula la mediana, los cuartiles y los bigotes a partir de los datos. La tabla completa incluye además `geom_col()` $\leftrightarrow$ `ax.bar()` para barras, `geom_hline()` $\leftrightarrow$ `ax.axhline()` para líneas horizontales de referencia, y `facet_wrap()` $\leftrightarrow$ `plt.subplots(nrows, ncols)` para paneles.

63. a V · b V · c F · d F.

Un gráfico se construye por capas superpuestas, y el gráfico de ozono mensual de la clase 7 tiene dos: una línea azul que une los cuatro valores y puntos rojos que los marcan. En `ggplot2` son `geom_line()` y `geom_point()` sumados al mismo `ggplot()`; en `matplotlib`, `ax.plot()` y `ax.scatter()` llamados sobre el mismo `ax`. Ambas capas representan los mismos cuatro datos; lo que cambia es la geometría. Las opciones a y b son verdaderas. El teórico de la clase 8 mostró las dos capas por separado para hacer visible que son cosas distintas.

La opción c es falsa. El orden de las capas determina cuál queda encima. En `ggplot2`, la última capa sumada se dibuja última y tapa a las anteriores donde se superponen. En `matplotlib` ocurre lo mismo con el orden de las llamadas, salvo que se indique explícitamente el argumento `zorder`: un valor mayor se dibuja encima. Por eso el script asigna `zorder=3` a `ax.scatter()`, para garantizar que los puntos rojos queden sobre la línea azul y no debajo de ella, donde la línea los taparía parcialmente.

La opción d es falsa. `theme_minimal()` en `ggplot2`, y la función `tema_minimal()` que el script de Python define para imitarlo, modifican la apariencia del gráfico: quitan el fondo gris, el marco, ajustan la grilla. No tocan los datos ni las geometrías. El tema es la capa de presentación, independiente del contenido.

64. a V · b V · c V · d F.

Los ejes y sus escalas son decisiones del analista, y el teórico de la clase 8 las examinó sobre el gráfico de ozono mensual.

El gráfico de partida tenía en el eje horizontal los números 1, 2, 3 y 4, porque la columna `mes` es numérica. Un lector externo tiene que deducir que son meses. Reemplazar las etiquetas por Ene, Feb, Mar, Abr elimina esa deducción. La opción a es verdadera. En `matplotlib` se hace con `ax.set_xticks([1, 2, 3, 4], ["Ene", "Feb", "Mar", "Abr"])`, que fija las posiciones de las marcas y las etiquetas que se muestran en ellas; en `ggplot2`, con `scale_x_continuous(breaks = 1:4, labels = c("Ene", "Feb", "Mar", "Abr"))`. La opción c es verdadera.

La opción b es verdadera y es el punto más importante. Cuando el eje vertical se ajusta automáticamente al rango de los datos, el gráfico de ozono empieza en 24 y termina en 36. En ese rango, la caída de febrero a marzo ocupa casi toda la altura del gráfico y parece un desplome. Con el eje desde cero, la misma caída es un descenso del 30 %: apreciable, no dramático. Ninguna de las dos escalas es falsa, pero la primera induce

una lectura exagerada. Healy (2018) trata este mecanismo como una de las formas más comunes de engañar con un gráfico sin alterar ningún dato.

La opción d es falsa por la misma razón que en la pregunta 57. `set_xticks()` cambia las etiquetas, `set_ylim()` cambia los límites; los datos permanecen intactos. La escala es una decisión de presentación, y como tal debe tomarse con conciencia de su efecto sobre el lector.

65. a $V \cdot b$ F $\cdot c$ V $\cdot d$ V.

Los textos de un gráfico son los que permiten que se entienda sin explicación adicional, y el teórico de la clase 8 fijó tres criterios.

El título debe decir qué se representa, dónde y cuándo. El gráfico de partida se titulaba “Concentración de Ozono en el ambiente, por mes, 2024”, que es aceptable pero impreciso: no dice que es la media mensual ni que el período es enero a abril. La versión corregida, “Ozono troposférico: media mensual, enero a abril de 2024”, responde las tres preguntas. La opción a es verdadera.

El rótulo del eje debe nombrar la variable y su unidad. “Valores” no hace ninguna de las dos cosas y es el ejemplo de lo que no debe escribirse; “Ozono ($\mu\text{g}/\text{m}^3$)” es el rótulo correcto. Las opciones b y d se refieren a esto: la b es falsa y la d es verdadera. La unidad no es un detalle: un lector que no la conoce no puede juzgar si 30 es mucho o poco, ni comparar el gráfico con la guía de la OMS, que está en microgramos por metro cúbico.

La fuente de los datos se indica en el gráfico, habitualmente como una nota al pie en letra pequeña. La versión corregida del script agrega `fig.text(..., "Fuente: ...")`. Un gráfico circula separado del documento que lo produjo, y sin la fuente nadie puede verificar de dónde salieron los datos. La opción c es verdadera.

66. a $V \cdot b$ V $\cdot c$ F $\cdot d$ V.

El script de la clase 9 encapsula el gráfico de ozono en una función, `grafico_ozono(datos, color_puntos="red", color_linea="blue", ylim=(0, 40))`, y la pregunta examina por qué conviene hacerlo.

La razón principal es que el código se escribe una vez y se invoca cuantas veces haga falta. El teórico necesitaba el mismo gráfico con puntos rojos y con puntos verdes; sin la función, habría que copiar el bloque de doce líneas y cambiar un argumento en la copia. Con la función, basta `grafico_ozono(ozono_mes, color_puntos="green")`. Las opciones a y b son verdaderas.

La opción d señala el problema que la función evita. Cuando el mismo bloque existe en dos lugares, toda corrección hay que hacerla en los dos, y tarde o temprano se hace en uno solo. A partir de ese momento las dos copias divergen: una tiene el rótulo corregido y la otra no, una empieza el eje en cero y la otra no. Con una función hay un único lugar que corregir. Es el mismo principio que justifica versionar el script de obtención y no los datos: una única fuente, sin duplicados que mantener sincronizados.

La opción c es falsa. Definir una función no afecta el tiempo de ejecución del gráfico; el código que se ejecuta es el mismo, solo que empaquetado con un nombre.

67. a $V \cdot b$ V $\cdot c$ V $\cdot d$ F.

Un gráfico que solo se muestra en pantalla no sirve para un informe. Hay que guardarlo en un archivo, y las dos bibliotecas ofrecen una función para hacerlo.

En `matplotlib`, `fig.savefig("g.png", dpi=300, bbox_inches="tight")` guarda la figura en el archivo indicado. El argumento `dpi` fija la resolución (300 es adecuado para impresión y documentos), y `bbox_inches="tight"` recorta los márgenes sobrantes. El formato se deduce de la extensión: `.png`, `.pdf`, `.svg`. La opción a es verdadera.

En `ggplot2`, `ggsave("g.png")` guarda el último gráfico producido, o el que se le pase como argumento `plot`. Admite `width`, `height` y `dpi`. La opción b es verdadera.

La opción c es verdadera y conecta con las reglas de Git de la clase 1. Las figuras se escriben en la carpeta `salidas/`, que el archivo `.gitignore` excluye del control de versiones. No se versionan porque se regeneran: quien clona el repositorio y ejecuta el script obtiene las mismas figuras. Versionarlas produciría un repositorio pesado y obligaría a confirmar una imagen nueva cada vez que se corrige un rótulo.

La opción d es falsa. Una captura de pantalla no es reproducible: nadie puede regenerarla ejecutando código, y no queda registro de qué datos ni qué parámetros la produjeron. Tampoco permite controlar la resolución ni el tamaño, y suele incluir elementos de la interfaz que no corresponden al gráfico.

68. a V · b V · c V · d F.

Para representar una serie en el tiempo, `matplotlib` necesita saber que el eje horizontal es temporal, y eso depende del tipo de la columna de fechas.

Si la columna es de tipo `datetime` —obtenida con `parse_dates` al leer, o con `pd.to_datetime()` después—, `matplotlib` la reconoce como fecha y construye un eje temporal: ubica cada observación en su instante real, elige marcas razonables (días, semanas, meses según el rango) y las rotula como fechas. La opción a es verdadera.

Si la columna quedó como texto, `matplotlib` no tiene forma de saber que "2024-03-07" es una fecha. La trata como una categoría más, como si fuera el nombre de una estación, y ubica las observaciones a intervalos iguales en el orden en que aparecen. Las consecuencias son dos: si faltan días, el gráfico no lo muestra, porque el 5 de marzo queda pegado al 8 de marzo sin hueco; y si hay muchas observaciones, el eje se llena de etiquetas de texto superpuestas e ilegibles. La opción b es verdadera.

La opción c es verdadera. `ax.plot(df["fecha"], df["o3"])` acepta directamente una columna `datetime` de `pandas`. Es lo que hace el script de la clase 7 al graficar `ozono_hora["fecha_hora"]` y `ozono_dia["dia"]`.

La opción d es falsa. No es necesario convertir la fecha a un número antes de graficar; `matplotlib` se encarga de esa conversión internamente. Hacerlo a mano produce un eje con números como 19789 en lugar de fechas.

69. a V · b V · c F · d V.

Cuando `matplotlib` construye un eje temporal, elige automáticamente el formato de las etiquetas, y ese formato no siempre es el deseado: puede mostrar el año completo donde bastaría el día, o mezclar formatos entre marcas. El módulo `matplotlib.dates` permite controlarlo.

Se importa con `import matplotlib.dates as mdates`; el alias `mdates` es la convención. La opción `a` es verdadera.

`mdates.DateFormatter("%d/%m")` crea un formateador que escribe cada fecha como día y mes separados por barra. Se aplica al eje con `ax.xaxis.set_major_formatter(...)`, y a partir de ese momento todas las marcas principales del eje horizontal se rotulan con ese formato. En el script de la clase 8, un panel usa `"%d/%m"` para mostrar 07/03 y otro usa `"%a %d"` para mostrar jue 07. Las opciones `b` y `d` son verdaderas.

La opción `c` es falsa. El formateador actúa sobre las etiquetas del eje, no sobre los datos. La columna de fechas conserva su tipo `datetime` y todos sus valores; lo único que cambia es el texto que se muestra en cada marca. Es la misma distinción que en las preguntas 57 y 64: una decisión de presentación que no altera el contenido.

Además del formateador, `matplotlib.dates` ofrece localizadores (`mdates.DayLocator()`, `mdates.HourLocator(interval=6)`) que controlan dónde se colocan las marcas. Formateador y localizador se usan juntos cuando el eje automático no da un resultado legible.

70. `a V · b V · c F · d V`.

Los códigos de formato de fecha son una convención heredada de la función `strftime` del lenguaje C, y por eso son los mismos en R, en pandas, en matplotlib y en la mayoría de los lenguajes.

`%d` es el día del mes con dos cifras (07), `%m` el mes con dos cifras (03), `%Y` el año con cuatro cifras (2024), `%H` la hora en formato de 24 horas (14), `%M` el minuto (05). Las opciones `a` y `b` son verdaderas. A ellos se suman `%S` para segundos, `%a` para el nombre abreviado del día de la semana, `%A` para el nombre completo, `%b` para el mes abreviado y `%B` para el mes completo.

La opción `c` es falsa. `%Y` y `%y` no son equivalentes: `%Y` produce el año con cuatro cifras (2024) y `%y` con dos (24). La distinción entre mayúscula y minúscula es significativa en todos los códigos; `%m` es mes y `%M` es minuto, y confundirlos produce fechas como 2024-14-03 que son errores difíciles de detectar. La opción `d` es verdadera. En R, `format(fecha, "%d/%m/%Y")` convierte una fecha a texto con ese formato. En pandas, `fecha.dt.strftime("%d/%m/%Y")` hace lo mismo con una columna. En matplotlib, `DateFormatter("%d/%m/%Y")` lo aplica a las etiquetas del eje. Y en la dirección inversa, los mismos códigos sirven para indicar cómo leer una fecha escrita como texto: `pd.to_datetime(x, format="%d/%m/%Y")` o `as.Date(x, format = "%d/%m/%Y")` cuando el formato del archivo no es el estándar año-mes-día.

71. `a V · b V · c V · d F`.

Antes de graficar una serie hay tres decisiones que el script de la clase 7 toma y que conviene hacer explícitas.

La primera es el nivel de agregación. El archivo de ozono tiene una medición por minuto durante cuatro meses: más de 170 000 filas por estación. Graficarlas todas produce una franja densa de la que no se distingue nada; la línea sube y baja miles de veces dentro del ancho de un píxel. Agregar por hora o por día reduce el número de puntos a unos miles o unos cientos, y hace visible la estructura. Pero la agregación

es una decisión de análisis, no un trámite: al agregar por día se pierde el ciclo diario, que es lo que muestra el perfil horario de la clase 9. La opción a es verdadera.

La segunda es cómo agregar. `pd.Grouper(key="fecha", freq="D")` dentro de un `groupby` agrupa por día calendario. Tras el `groupby().agg()`, la fecha queda como índice del resultado; `reset_index()` la devuelve como columna, que es lo que `ax.plot()` espera. Las opciones b y c son verdaderas.

La tercera es el orden. La opción d es falsa. `ax.plot()` no ordena los datos: une los puntos en el orden en que aparecen en las columnas que recibe. Si el archivo tiene las fechas desordenadas —porque se concatenaron varios archivos, porque la fuente los exporta por estación—, la línea va del 3 de marzo al 1 de marzo y luego al 8, cruzándose sobre sí misma. El gráfico se ve como una maraña y no hay mensaje de error. Hay que ordenar antes con `sort_values("fecha")` en pandas o `arrange(fecha)` en R. El script de la clase 9 lo hace explícitamente antes de calcular rachas y rezagos.

Clase 9 — Unión de fuentes y gráficos por grupo

72. a V · b V · c F · d V.

Unir dos tablas es poner en la misma fila información que proviene de fuentes distintas. Para hacerlo hay que indicar con qué se aparean las filas: la clave de unión. En el ejemplo del curso, una medición de ozono y una de PM2.5 corresponden al mismo evento si fueron tomadas en la misma estación y en el mismo instante; la clave es la combinación de estación y fecha. La opción b es verdadera.

`inner_join(ozono, pm, by = c("estacion", "fecha"))` en R, o `pd.merge(ozono, pm, on=["estacion", "fecha"], how="inner")` en pandas, conservan únicamente las filas cuya clave existe en las dos tablas. Un minuto en que hay ozono pero no PM2.5, o una estación que solo mide uno de los dos, queda fuera del resultado. La opción a es verdadera. Otras variantes de la unión se comportan de forma distinta: `left_join` conserva todas las filas de la tabla izquierda y completa con faltantes las que no tienen correspondencia; `full_join` conserva todas las filas de ambas.

La opción c es falsa. Las filas excluidas de la unión no se borran de ningún lado. Las tablas ozono y pm permanecen completas en memoria y en el disco; la unión produce una tabla nueva, aire, sin modificar las originales.

La opción d es verdadera y es la lección de la sección 2 del script. Lo que quedó fuera de la unión sigue disponible para graficar: las series de ozono de todas las estaciones se dibujan desde ozono, y las de PM2.5 desde pm. Lo que no se puede hacer con esas filas es compararlas con la otra variable en el mismo instante, porque no hay con qué compararlas. La tabla aire se usa recién cuando se quiere poner una variable contra la otra, y por eso el gráfico de dispersión de la sección 7 solo tiene la estación en común. Al unir hay que mirar cuántas filas se pierden y por qué; una unión que descarta el 90 % de los datos sin que nadie lo note es un error de análisis, no de código.

73. a V · b V · c F · d V.

En `ggplot2`, un atributo visual —color, tamaño, forma— puede fijarse o mapearse, y la diferencia está en si va dentro o fuera de `aes()`.

`geom_line(color = "blue")`, con el color fuera de `aes()`, fija el atributo: toda la línea es azul. Es una decisión de legibilidad, sin contenido informativo: el azul no representa ninguna variable. Por eso no produce leyenda; no habría nada que explicar en ella. La opción a es verdadera y la c es falsa.

`aes(color = estacion)`, con el color dentro de `aes()`, mapea el atributo a una variable: el color pasa a ser informativo, porque cada valor de `estacion` recibe un color distinto. `ggplot2` parte los datos por los valores de la variable, dibuja una línea por grupo con su color, y genera automáticamente la leyenda que dice qué color corresponde a qué estación. La opción b es verdadera. No hace falta escribir un bucle que recorra las estaciones; el mapeo lo hace. La opción d es verdadera.

En `matplotlib` no existe esa distinción automática: para obtener un color por estación hay que recorrer los grupos con `for est, d in datos.groupby("estacion")`: `ax.plot(d["fecha"], d["o3"], label=est)` y luego llamar a `ax.legend()`. El script de la clase 9 lo hace así. La comparación entre los dos enfoques muestra lo que la gramática de `ggplot2` aporta: la separación entre qué se representa (el mapeo) y cómo se dibuja (la geometría).

74. a V · b V · c F · d V.

La clase 9 presentó tres formas de organizar más de una serie en un gráfico, y dos formas de combinar tablas que las preceden.

`facet_wrap(~ estacion)` divide el gráfico en paneles, uno por cada valor de la variable, todos con la misma estructura. Cada panel responde la misma pregunta para un grupo distinto. Es la alternativa al color cuando hay muchos grupos o cuando las series se superpondrían demasiado. En `matplotlib`, el equivalente es `plt.subplots(nrows, ncols)` con un bucle que dibuja en cada eje. La opción a es verdadera.

Para poner en paneles el ozono y el PM2.5, que están en tablas distintas, hay que apilarlas primero: una tabla con todas las filas de ambas y una columna adicional que indique de cuál proviene cada fila. `bind_rows()` en R y `pd.concat()` en pandas hacen exactamente eso: colocan una tabla debajo de la otra, alineando columnas por nombre. La opción b es verdadera.

La opción c es falsa. `pd.merge()` no apila: une por clave, poniendo en la misma fila información de las dos tablas, como en la pregunta 72. Apilar y unir son operaciones distintas: apilar agrega filas; unir agrega columnas. Confundirlas es un error frecuente porque ambas “juntan tablas”.

`patchwork` es un paquete de R que combina gráficos de `ggplot2` ya construidos. El operador `/` los apila verticalmente y el operador `|` los coloca lado a lado. En la clase 9 se usó `/` para poner la serie de ozono sobre la de PM2.5, porque tienen unidades y rangos distintos y compartir el eje vertical aplastaría una de las dos. La opción d es verdadera.

75. a V · b V · c F · d V.

Las secciones 5 a 8 del script de la clase 9 hacen tres preguntas distintas a los mismos datos.

El perfil horario agrupa por la hora del día, de 0 a 23, no por cada hora del calendario.

Cada punto del perfil es la media de todas las 14:00 del período, unos 120 días. Responde a qué hora del día hay más ozono, y la respuesta es la tarde, cuando hay sol; y a qué hora hay más PM2.5, la noche y la madrugada, cuando el aire está quieto. La serie temporal responde otra cosa: cuándo, en qué fecha, ocurrió cada valor. Son agrupamientos distintos para preguntas distintas. La opción a es verdadera.

El diagrama de caja por mes muestra la distribución completa de las medias diarias: mediana, cuartiles, extremos y atípicos. La media sola, que es lo que muestra el gráfico de líneas mensual de la clase 7, esconde esa distribución: dos meses con la misma media pueden tener dispersiones muy distintas. La opción b es verdadera y la c es falsa.

El gráfico de dispersión de ozono contra PM2.5 mostró tres cosas. Una pared vertical en $PM2.5 \approx 3$: muchos puntos alineados en ese valor, cualquiera sea el ozono. No es un fenómeno atmosférico; es el límite de detección del sensor, que no puede medir por debajo de ese valor y lo reporta como piso. La opción d es verdadera. Además, las dos variables son asimétricas a la derecha, y la nube tiene forma de triángulo y no de recta: cuando una es alta, la otra no. Coloreando los puntos por hora del día se ve que los extremos de cada variable ocurren a horas distintas, lo que explica el triángulo.

Clase 10 — Series de tiempo: una primera mirada

76. a V · b V · c F · d V.

Una serie de tiempo es una columna cuyo índice es el tiempo, con observaciones a paso fijo. El paso fijo —la grilla regular— es lo que da sentido a las operaciones de rezago: “el valor anterior” significa siempre lo mismo (un minuto antes, diez minutos antes) y la autocorrelación a rezago k se refiere a un intervalo de tiempo definido. Si las observaciones están a intervalos irregulares, el rezago de una posición corresponde a veces a un minuto y a veces a una hora, y la autocorrelación calculada no mide nada interpretable. La opción a es verdadera.

Los datos del sensor tienen huecos, así que hay que construir la grilla. `df.resample("10min").mean()` toma una serie con índice `datetime`, la divide en intervalos de diez minutos y calcula la media de cada uno; los intervalos sin datos quedan como faltantes. El resultado tiene una fila cada diez minutos, sin excepción. La opción d es verdadera.

Se eligió un paso de diez minutos y no de una hora. Al promediar sesenta minutos se suaviza la serie: los picos cortos desaparecen y la variabilidad que interesa para la autocorrelación se pierde. El paso corto conserva esa variabilidad y permite ver, por ejemplo, que el primer pico de autocorrelación del ozono está cerca de las 24 horas. La opción b es verdadera y la c es falsa: el paso adecuado depende del fenómeno y de la pregunta, y para estudiar la estructura temporal fina, el paso horario es demasiado grueso.

77. a V · b V · c F · d V.

El rezago es la operación básica del análisis de series: desplazar la serie k posiciones para alinear cada valor con el que ocurrió k pasos antes. `x.shift(1)` en pandas y `lag(x)` en dplyr desplazan un paso: la fila de las 10:10 recibe el valor de las 10:00. La primera fila queda sin valor anterior y se completa con faltante. La opción a es verdadera.

El gráfico de rezago dibuja cada valor contra el anterior: x_t en el eje vertical, x_{t-1} en el horizontal. Si los puntos se concentran sobre la diagonal, cada valor es parecido al anterior: la serie tiene autocorrelación positiva fuerte, que es lo que se observa en el ozono a diez minutos. Si los puntos forman una nube sin estructura, el valor anterior no informa nada sobre el actual. La opción b es verdadera.

La autocorrelación es la correlación de una serie consigo misma desplazada k pasos: `x.corr(x.shift(k))`. El prefijo *auto* indica que es una sola serie; la correlación entre dos series distintas es la correlación cruzada de la pregunta siguiente. La opción c es falsa.

La función de autocorrelación (ACF) calcula ese coeficiente para muchos valores de k y los grafica contra k . El script la construye con una comprensión de lista, `[suave["o3"].corr(suave["o3"].shift(k)) for k in rezagos]`, sin recurrir a paquetes especializados. La forma de la ACF muestra la estructura temporal: un decaimiento lento indica persistencia; picos a intervalos regulares indican un ciclo, y en el ozono aparecen a 24, 48 horas, el ciclo diario. La opción d es verdadera.

78. a $V \cdot b$ $V \cdot c$ $F \cdot d$ V .

La correlación cruzada extiende la idea de la autocorrelación a dos series: mide la correlación entre una serie y la otra desplazada k pasos, para distintos valores de k , positivos y negativos. La opción a es verdadera.

En el curso se calculó entre ozono y PM2.5 en la estación Curva de Maroñas, que es la única en que las dos redes de medición coinciden y, por lo tanto, la única en que las dos series pueden alinearse minuto a minuto. La opción b es verdadera.

Si la correlación cruzada alcanza su máximo en $k = 0$, las dos series suben y bajan al mismo tiempo. Si el máximo está en un k distinto de cero, una serie se adelanta a la otra en k pasos: los movimientos de una preceden a los movimientos de la otra. En el ejemplo, el desfase refleja que el ozono tiene su máximo por la tarde y el PM2.5 por la noche. La opción d es verdadera.

La opción c es falsa y conecta con las lecturas de Bueno de Mesquita y Fowler y de Hernán, Hsu y Healy. Una correlación cruzada alta, incluso con un desfase claro, no implica que una serie cause la otra. Ambas pueden responder a un mismo factor común —el ciclo diario de radiación solar, temperatura y tráfico— que mueve a las dos con distinto retardo. Establecer causalidad requiere un diseño o supuestos que la correlación por sí sola no aporta.

Lecturas obligatorias

79. a $V \cdot b$ $V \cdot c$ $V \cdot d$ F .

The Future of Data Analysis (1962) es el texto en que Tukey propone que el análisis de datos es una disciplina propia, distinta de la estadística matemática que dominaba la formación de los estadísticos de su época. Su argumento es que el análisis de datos es una ciencia empírica: se aprende haciendo, se juzga por sus resultados sobre datos reales, y no se reduce a la deducción de propiedades óptimas bajo supuestos. La opción a es verdadera.

De esa posición se desprende la revalorización de la exploración. Para Tukey, mirar los datos, graficarlos, resumirlos, buscar estructura inesperada, no es un paso preliminar al análisis “verdadero” sino parte del análisis. Su obra posterior sobre análisis exploratorio de datos desarrolla esa idea. La opción c es verdadera. La opción d es falsa: limitar el análisis a la verificación de hipótesis fijadas de antemano es precisamente el modelo que Tukey critica, porque deja fuera todo lo que los datos podrían enseñar y que nadie pensó en preguntar.

La opción b cita la frase más conocida del artículo: es preferible una respuesta aproximada a la pregunta correcta, que muchas veces es vaga, que una respuesta exacta a la pregunta equivocada, que siempre puede formularse con precisión. Es una advertencia contra la tentación de elegir el problema por la elegancia del método en lugar de por su relevancia. La opción b es verdadera. El curso está construido sobre esta idea: cada unidad empieza por el tipo de pregunta y no por la técnica.

80. a V · b V · c V · d F.

Statistical Modeling: The Two Cultures (2001) describe dos maneras de usar modelos estadísticos para extraer información de los datos.

La cultura de los modelos de datos supone que los datos fueron generados por un modelo estocástico dado —una regresión lineal, un modelo logístico— y se ocupa de estimar sus parámetros e interpretarlos. Es la cultura dominante en la estadística académica. La opción a es verdadera.

La cultura de los modelos algorítmicos considera que el mecanismo que genera los datos es complejo y desconocido —una caja negra— y no intenta describirlo; busca un algoritmo que prediga bien la salida a partir de las entradas, y evalúa los modelos por su exactitud predictiva sobre datos nuevos. Es la cultura del aprendizaje automático. La opción b es verdadera.

Breiman sostiene que la primera cultura, al comprometerse con un modelo antes de mirar los datos, produce conclusiones sobre el modelo y no sobre la naturaleza, y que la segunda ha resuelto problemas que la primera no pudo. La caja negra que el curso presentó en la clase 2, con la dinámica de los objetos ocultos, es la de este artículo. La opción c es verdadera.

La opción d es falsa. El artículo se publicó en *Statistical Science* con comentarios de D. R. Cox, Bradley Efron, Bruce Hoadley y Emanuel Parzen, y una réplica de Breiman. Los comentaristas son autores de sus comentarios, no del artículo, y por eso la referencia correcta es “Breiman, L. (2001)” y no “Breiman et al. (2001)”. Los comentarios figuran en la bibliografía complementaria del curso como lectura aparte.

81. a V · b V · c F · d V.

A Second Chance to Get Causal Inference Right (2019) propone una clasificación de las tareas que la ciencia de datos puede abordar, y sostiene que la disciplina ha descuidado una de ellas.

Las tres tareas son la descripción (resumir cómo son los datos: cuántos, cuánto, con qué distribución), la predicción (estimar el valor de una variable a partir de otras, en datos nuevos generados por el mismo proceso) y la inferencia causal (estimar qué

ocurriría con una variable si se interviniera sobre otra: qué pasaría con las ventas si se bajara el precio). La opción a es verdadera.

Las dos primeras tareas pueden resolverse con los datos y algoritmos; la tercera no. Predecir bien no dice nada sobre el efecto de una intervención, porque la predicción explota las asociaciones existentes en los datos, y esas asociaciones pueden deberse a factores comunes, a causalidad inversa o a selección. Un modelo que predice muy bien las ventas a partir del precio puede estar capturando que los precios suben cuando hay más demanda, y no que el precio cause las ventas. Para responder la pregunta causal hacen falta supuestos sobre el mecanismo —qué variables confunden, qué no— que provienen del conocimiento del dominio y no pueden extraerse de los datos. Las opciones b y d son verdaderas; la c es falsa.

El curso dedica su unidad 8 a esta tarea, con experimentos y diferencias en diferencias; la lectura se asigna desde el comienzo para que la distinción entre asociación y causalidad esté presente en todo el trabajo con datos observacionales.

82. a V · b V · c V · d F.

Thinking Clearly with Data (2021) es un manual de razonamiento cuantitativo para lectores sin formación estadística avanzada, organizado alrededor de los errores más frecuentes al interpretar datos.

El primero es tomar una correlación por una relación causal. Los autores presentan sistemáticamente las alternativas: una variable de confusión que causa ambas, causalidad inversa (el efecto causa la causa supuesta), y azar. Sin descartar esas alternativas, una correlación, por fuerte que sea, no autoriza a afirmar un efecto. La opción a es verdadera y la d es falsa.

El segundo es la relación entre correlación y variación. Para que dos variables estén correlacionadas, ambas tienen que variar; si la variable explicativa es constante en la muestra, no hay correlación que estimar. Parece obvio, pero es la fuente de muchos errores: estudiar el efecto de una política solo en los lugares donde se aplicó, o el efecto de una característica que todos los casos comparten. La opción b es verdadera.

El tercero es la selección sobre la variable dependiente. Estudiar solo las empresas exitosas para descubrir qué las hizo exitosas, o solo los pacientes que sobrevivieron para identificar qué los salvó, produce conclusiones sesgadas porque se excluyó de la muestra a los casos que permitirían la comparación. La opción c es verdadera.

El libro fundamenta la unidad 8 del curso, junto con Hernán, Hsu y Healy, y su lectura completa es obligatoria porque los errores que describe aparecen en cualquier análisis observacional, incluidos los de los proyectos de equipo.

83. a V · b V · c V · d F.

Wes McKinney es el creador de pandas, y *Python for Data Analysis* (3.^a edición, 2022) es el manual de referencia escrito por su autor. La opción a es verdadera y la d es falsa: el libro trata de Python.

El libro presenta las dos estructuras centrales de pandas. Series es una columna con índice; DataFrame es una tabla de columnas alineadas por un índice común. Todo lo

demás —selección, filtrado, agregación— se construye sobre ellas. La opción b es verdadera.

Los capítulos que el curso utiliza son los de lectura y escritura de archivos (CSV, Excel, JSON), limpieza y preparación (faltantes, duplicados, transformación de texto), agrupación con groupby y agregación, y series de tiempo (índices datetime, resample, shift, ventanas móviles). También incluye capítulos sobre NumPy, que es la base de pandas, y sobre visualización con matplotlib. La opción c es verdadera.

Es el libro que corresponde consultar cuando una operación de pandas no hace lo esperado: la documentación oficial es más completa pero menos didáctica, y las respuestas de los asistentes de inteligencia artificial pueden estar desactualizadas o ser incorrectas. La tercera edición está disponible en línea sin costo en el sitio del autor.

84. a V · b V · c V · d F.

Python Data Science Handbook (2.^a edición, 2023), de Jake VanderPlas, cubre las cuatro bibliotecas que constituyen el núcleo de la ciencia de datos en Python: NumPy para arreglos numéricos, pandas para tablas, Matplotlib para gráficos y scikit-learn para aprendizaje automático. La opción a es verdadera y la d es falsa: no trata de SQL.

Los capítulos sobre Matplotlib son la referencia del curso para la construcción de gráficos en Python. Explican la interfaz orientada a objetos (fig, ax = plt.subplots(), ax.plot(), ax.set()) que se usa en todos los scripts, los tipos de gráfico, la personalización de ejes y leyendas, y los paneles múltiples. La opción b es verdadera.

La parte dedicada a scikit-learn queda fuera del alcance de esta materia. El aprendizaje automático se desarrolla en otra asignatura de la carrera, y el repositorio del curso no incluye scikit-learn entre sus dependencias: todo lo que se hace se resuelve con NumPy, pandas, SciPy y statsmodels. La opción c es verdadera. La lectura de esos capítulos no está prohibida, pero no forma parte de lo que se evalúa.

Como el libro de McKinney, está disponible en línea sin costo en el sitio del autor, en forma de cuadernos ejecutables.

85. a V · b V · c V · d F.

R for Data Science (2.^a edición, 2023), de Hadley Wickham, Mine Çetinkaya-Rundel y Garrett Grolmund, es el manual de referencia del tidyverse y el libro de R del curso. La opción d es falsa.

El libro organiza el trabajo con datos en un flujo que presentó la clase 5 con la figura del “whole game”: importar los datos, ordenarlos (*tidy*), y luego un ciclo de transformar, visualizar y modelar que se repite hasta comprender, para finalmente comunicar. El ciclo es iterativo: cada visualización sugiere una transformación, cada modelo sugiere una visualización. La opción a es verdadera.

El concepto de datos ordenados es central. Una tabla está ordenada cuando cada variable ocupa una columna, cada observación ocupa una fila y cada valor ocupa una celda. Muchas tablas del mundo real no cumplen esa condición: tienen un mes por columna en lugar de una columna mes, o dos variables en una celda. Las funciones

`pivot_longer()` y `pivot_wider()` del paquete `tidyr` convierten entre formatos, y el script de la clase 7 usa `pivot_longer()` para poner media, p95 y máximo en una sola columna antes de graficarlos por color. La opción b es verdadera.

El libro usa los paquetes del tidyverse: `ggplot2` para gráficos, `dplyr` para transformación, `readr` para lectura, `tidyr` para reorganización, `lubridate` para fechas, `stringr` para texto. Son los mismos paquetes de los scripts de R del curso. La opción c es verdadera. Está disponible en línea sin costo, en inglés y en traducción al español.

86. a V · b V · c V · d F.

Data Visualization: A Practical Introduction (2018), de Kieran Healy, combina fundamentos de percepción visual con instrucción práctica en `ggplot2`.

Su primer capítulo resume la investigación sobre percepción de gráficos: las personas juzgan con mayor precisión las posiciones sobre un eje común (dónde está un punto, qué altura tiene una barra), con menos precisión las longitudes sin eje común y los ángulos, y con mucha menos precisión las áreas y las intensidades de color. De ahí se sigue una jerarquía: cuando el mensaje depende de comparar valores con exactitud, hay que codificarlos como posición, y reservar el área y el color para información secundaria. La opción a es verdadera.

De esa jerarquía se deriva la crítica a los gráficos de torta, que codifican los valores como ángulos y áreas, y a los gráficos en tres dimensiones, que distorsionan las áreas por la perspectiva. Healy los desaconseja porque exigen al lector la operación perceptiva menos precisa. La opción d es falsa.

El libro dedica atención a cómo un gráfico induce a error sin datos falsos: ejes que no empiezan en cero, escalas truncadas, razones de aspecto que exageran o aplastan una tendencia. Es el mecanismo de la pregunta 64. La opción b es verdadera. Todos los ejemplos están contruidos con `ggplot2`, con el código incluido, y el libro funciona como tutorial del paquete. La opción c es verdadera.

87. a V · b V · c V · d F.

Storytelling with Data (2015), de Cole Nussbaumer Knaflic, trata la visualización desde el punto de vista de la comunicación: cómo construir un gráfico para que un lector con poco tiempo entienda una afirmación y actúe en consecuencia.

Su premisa es que un gráfico exploratorio y un gráfico para comunicar son cosas distintas. En la exploración se hacen muchos gráficos para descubrir qué hay en los datos; en la comunicación se hace uno, que sostiene una afirmación concreta. “Mostrar los datos” no es un objetivo: el objetivo es que el lector vea lo que el analista encontró. La opción a es verdadera.

De esa premisa se sigue la recomendación de eliminar todo lo que no contribuye al mensaje: bordes, grillas innecesarias, marcadores de datos redundantes, colores decorativos, leyendas que podrían reemplazarse por etiquetas directas. Cada elemento que no aporta compite por la atención del lector con lo que sí importa. La opción b es verdadera.

El título del gráfico, en esta perspectiva, no describe el tema (“Ozono por mes”) sino que enuncia el mensaje (“El ozono cayó un tercio entre febrero y marzo”). El lector

que solo lea el título ya sabe qué concluir. La opción c es verdadera.

El color se trata como un recurso escaso. Un gráfico con muchos colores no dirige la atención a ningún lado; un gráfico en gris con un solo elemento destacado en color lleva la mirada exactamente adonde el analista quiere. La opción d es falsa. Estas recomendaciones se aplican en el curso a las figuras de los informes finales, no a los gráficos de trabajo intermedios.